Finch: Sparse and Structured Tensor Programming with Control Flow

ANONYMOUS AUTHOR(S)

From FORTRAN to NumPy, tensors have revolutionized how we express computation. However, tensors in these, and almost all prominent systems, can only handle dense rectilinear integer grids. Real world tensors often contain underlying structure, such as sparsity, runs of repeated values, or symmetry. Support for structured data is fragmented and incomplete. Existing frameworks limit the tensor structures and program control flow they support to better simplify the problem.

In this work, we propose a new programming language, Finch, which supports *both* flexible control flow and diverse data structures. Finch facilitates a programming model which resolves the challenges of computing over structured tensors by combining control flow and data structures into a common representation where they can be co-optimized. Finch automatically specializes control flow to data so that performance engineers can focus on experimenting with many algorithms. Finch supports a familiar programming language of loops, statements, ifs, breaks, etc., over a wide variety of tensor structures, such as sparsity, run-length-encoding, symmetry, triangles, padding, or blocks. Finch reliably utilizes the key properties of structure, such as structural zeros, repeated values, or clustered non-zeros. We show that this leads to dramatic speedups in operations such as SpMV and SpGEMM, image processing, and graph analytics.

CCS Concepts: • **Software and its engineering** → **Control structures**; **Data types and structures**; *Imperative languages*; • **Mathematics of computing** → **Mathematical software**.

Additional Key Words and Phrases: Sparse Tensor, Structured Tensor, Control Flow, Programming Language

ACM Reference Format:
Anonymous Author(s). 2024. Finch: Sparse and Structured Tensor Programming with Control Flow. *J. ACM* 37, 4, Article 111 (August 2024), 58 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Arrays are the most fundamental abstraction in computer science. Arrays and lists are often the first-taught datastructure [4, Chapter 2.2], [61, Chapter 2.2]. Arrays are also universal across programming languages, from their introduction in Fortran in 1957 to present-day languages like Python [10], keeping more-or-less the same semantics. Modern array programming languages such as NumPy [50], SciPy [97], MatLab [73], TensorFlow [2], PyTorch [76], and Halide [78] have pushed the limits of productive data processing with arrays, fueling breakthroughs in machine learning, scientific computing, image processing, and more.

The success and ubiquity of arrays is largely due to their simplicity. Since their introduction, multidimensional arrays have represented dense, rectilinear, integer grids of points. By **dense**, we mean that indices are mapped to value via a simple formula relating multidimensional space to linear memory. Consequently, dense arrays offer extensive compiler optimizations and many convenient interfaces. Compilers understand dense computations across many programming constructs, such as for and while loops, breaks, parallelism, caching, prefetching, multiple outputs, scatters, gathers, vectorization, loop-carry-dependencies, and more. A myriad of optimizations have been developed for dense arrays, such as loop fusion, loop tiling, loop unrolling, and loop interchange. However, while dense arrays are the easiest way to program for performance, real world applications often require more complex data structures to reach peak efficiency.

Our world is full of structured data. In this work, we make the distinction between a **tensor**, which describes any multidimensional object which relates tuples of integer coordinates to values,

2024. ACM 0004-5411/2024/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

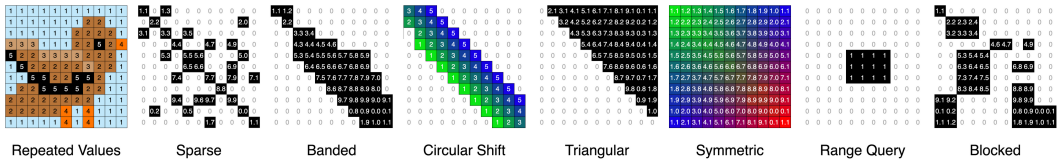


Fig. 1. A few examples of matrix structures arising in practice

such as vectors or matrices, and an **array**, the previously described classical data structure. We say that a tensor is **structured** when it has patterns that allows us to optimize storage or computation of the tensor. Sparse tensors (which store only nonzero elements) describe networks, databases, and simulations [5, 12, 16, 71]. Run-length encoding describes images, masks, geometry, and databases (such as a list of transactions with the date field all the same) [45, 84]. Symmetry, bands, padding, and blocks arise due to modeling choices in scientific computing (e.g., higher order FEMs) as well as in intermediate structures in many linear solvers (e.g., GMRES) [25, 75, 80]. Combinations of sparse and blocked matrices are increasingly under consideration in machine learning [31]. Even complex operators can be expressed as structured tensors. For example, convolution can be expressed as a matrix multiplication with the Toeplitz matrix of all the circular shifts of the filter [90].

Currently, support for structured data is fragmented and incomplete. Experts must hand write variations of even the simplest kernels, like matrix multiply, for each data structure/data set and architecture to get performance. Implementations must choose a small set of features to support well, resulting in a compromise between **program flexibility** and **data structure flexibility**. Hand-written solutions are collected in diverse libraries like MKL, OpenCV, LAPACK or SciPy [9, 23, 77, 97]. However, libraries will only ever support a subset of programs on a subset of data structure combinations. Even the most advanced libraries, such as GraphBLAS, which support a wide variety of sparse operations over various semi-rings always lack support for other features, such as N-D tensors, fused outputs, or runs of repeated values [24, 70]. While dense tensor compilers support an enormous variety of program constructs like early break and multiple left hand sides, they only support dense tensors [47, 78]. Special-purpose compilers like TACO [59], Taichi [53], StructTensor [44], or CoRa [39] which support a select subset of structured data structures (only sparse, or only ragged tensors) must compromise by greatly constraining the classes of programs which they support, such as tensor contractions. This trade-off is visualized in Tables 1 and 2.

Prior implementations are incomplete because the abstractions they use are tightly coupled with the specific data structures that they support. For example, TACO merge lattices represent Boolean logic over sets of non-zero values on an integer grid [60]. The polyhedral model allows various compilers to represent dense computations on affine regions [47]. Taichi enriches single static assignment form with a specialized instruction for accessing only a single sparse structure, but it supports more control flow [53]. These systems tightly couple their control flow to narrow classes

Feature / Tool	Halide	Taco	Cora	Taichi	Stur	Finch
Einsums/Contractions	✓	✓	✓	✓	✓	✓
Multiple LHS	✓		✓	✓		✓
Affine Indices	✓			✓	✓	✓
Recurrence	✓					
If-Conditions and Masks	✓	✓		✓		✓
Scatter Gather	✓			✓		✓
Early Break		✓		✓		✓
Unrestricted Read/Write	✓					

Table 1. Control flow support across various tools.

Feature / Tool	Halide	Taco	Cora	Taichi	Stur	Finch
Dense	✓	✓	✓	✓	✓	✓
Padded	✓					✓
One Sparse Operand		✓		✓		✓
Multiple Sparse Operands		✓				✓
Run-length						✓
Symmetric					✓	✓
Regular Sparse Blocks		✓				✓
Irregular Sparse Blocks						✓
Ragged			✓			✓

Table 2. Data structure support across various tools. Finch supports **both** complex programs and complex data structures.

of data structures to avoid the challenges that occur when we intersect complex control flow with structured data. There are two challenges:

Optimizations are specific to the indirection and patterns in data structures: These structures break the simple mapping between tensor elements and where they are stored in memory. For example, sparse tensors store lists of which coordinates are nonzero, whereas run-length-encoded tensors map several pixels to the same color value. These zero regions or repeated regions are optimization opportunities, and we must adapt the program to avoid repetitive work on these regions by referencing the stored structure.

Performance on structured data is highly algorithm dependent: The landscape of implementation decisions is dramatically unpredictable. For example, the asymptotic performance of sparse matrix multiplication can be impacted by the distribution of nonzeros, the sparse format, and the loop order [8, 105]. This means that performance engineering for such kernels requires the exploration of a large design space, changing the algorithm as well as the data structures.

In this work, we propose a new programming language, Finch, which supports *both* flexible control flow and diverse data structures. Finch facilitates a programming model which resolves the challenges of computing over structured tensors by **combining control flow and data structures into a common representation where they can be co-optimized**. In particular, Finch automatically specializes the control flow to the data so that performance engineers can focus on experimenting with many algorithms. Finch supports a familiar programming language of loops, statements, if conditions, breaks, etc., over a wide variety of tensor structures, such as sparsity, run-length-encoding, symmetry, triangles, padding, or blocks. This support would be useless without the appropriate level of structural specialization; Finch reliably utilizes the key properties of structure, such as structural zeros, repeated values, or clustered non-zeros.

As an example, a programmer might explore different ways to intersect only the even integers of two lists (represented as sparse vectors with sorted indices). The control flow here is only useful if the first example differs from the next two in that it actually selects only even indices as the two integer lists are merged and different from the last in that it does not require another tensor:

```
for i = _
    if i % 2 == 0
        c[i]=a[i]*b[i]
```

```
for i = _
    if i % 2 == 0
        ap[i] = a[i]
for i = _
    c[i] = ap[i] * b[i]
```

```
for i = _
    cp[i] = a[i] * b[i]
for i = _
    if i % 2 == 0
        c[i] = cp[i]
```

```
for i = _
   if i % 2 == 0
       f[i] = 1
for i = _
   c[i] = a[i] * b[i] * f[i]
```

1.1 Contributions

(1) More complex tensor structures than ever before. We are the first to extend level-by-level hierarchical descriptions to capture banded, triangular, run-length-encoded, or sparse datasets, and any combination thereof. We have chosen a set of level formats that completely captures all combinations of relevant structural properties (zeros, repeated values, and/or blocks). Although many systems (TACO, Taichi, SPF, Ebb) [17, 30, 53, 89] feature a flexible structure description, our level abstraction is more capable and extensible because it uses looplets [7] to express the structure of each level.

(2) A rich structured tensor programming language with for-loops and complex control flow constructs at the same level of productivity of dense tensors. To our knowledge, the Finch programming language is the first to support if-conditions, early breaks, and multiple left hand sides over structured data, as well as complex accesses such as affine indexing or scatter/gather of sparse or structured operands.

(3) A compiler that specializes programs to data structures automatically, facilitating an expressive language that makes it easier to search the complex space of algorithms and data structures. Finch reliably utilizes four key properties of structure, such as structural zeros, repeated values, clustered non-zeros, and singletons.

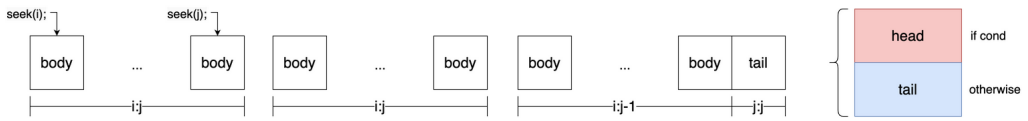
(4) Our compiler is highly extensible, evidenced by the variety of level formats and control flow constructs that we implement in this work. For example, Finch has been extended to support real-valued tensor indices with continuous tensors. Finch is also used as a compiler backend for the Python PyData/Sparse library [3].
(5) We evaluate the efficiency, flexibility, and expressiveness of our language in several case studies on a wide range of applications, demonstrating speedups over the state of the art in classic operations such as SpMV (geomean 1.26×, max 3.04×) and SpGEMM (geomean 1.30×, max 1.62×), to more complex applications such as graph analytics (geomean 2.47× on Bellman-Ford, reducing lines of code by 4× over GraphBLAS), and image processing (19.5× on the humansketches dataset [38]).

2 BACKGROUND

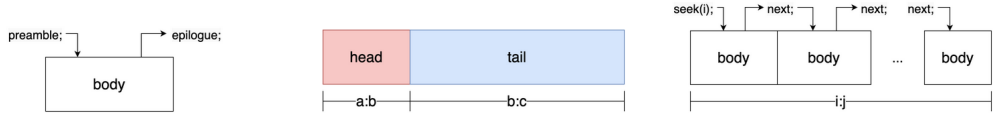
2.1 Looplets

Finch represents iteration patterns using looplets, a language that decomposes datastructure iterators hierarchically. Looplets represent the control-flow structures needed to iterate over any given datastructure, or multiple datastructures simultaneously. Because looplets are compiled with progressive lowering, structure-specific mathematical optimizations such as integrals, multiply by zero, etc. can be implemented using simple compiler passes like term rewriting and constant propagation during the intermediate lowering stages.

The looplets are described in Figure 2. We simplify the presentation to focus on the semantics, rather than precise implementation. For more background on looplets, we recommend the original work [7]. Several looplets introduce or modify variables in the scope of the target language. This allows looplets to lift code to the highest possible loop level. It is assumed that if a looplet introduces a variable, the child looplet will not modify that variable.



(a) A **Lookup** looplet represents a randomly accessible sequence as a function of the index.
(b) A **Run** looplet represents a sequence of many of the same value, usually stored once.
(c) A **Spike** looplet represents a run followed by a single, different value at the end of the target region.
(d) A **Switch** looplet represents a choice between different looplets under different conditions.



(e) A **Thunk** looplet allows us to add side effects such as caching a value for sublooplets to use.
(f) A **Sequence** looplet represents a sequence of a few different looplets, one after the other.
(g) A **Stepper** looplet represents a sequence of an unbounded number of identical looplets.

Fig. 2. The looplet language, as understood in a correct execution of a Finch program.

Finch advances the state-of-the-art over the looplets work [7]. While looplets presented a way to merge iterators over single dimensional structures, Finch is the only framework to support such a broad range of multi-dimensional structured data in a programming language with fully-featured control flow. Looplets provide a powerful mechanism to simplify structured loops, but our paper shows how to make this functionality practical; Finch uses looplets as a symbolic loop simplification

engine. The precise choice and implementation of tensor level structures, the lifecycle interface between levels and looplets, and the canonicalization of fancy indexing and masking all serve to utilize and recombine looplets to achieve efficient computation over structured tensors.

Description	Arguments
lookup(body): The Lookup looplet represents a randomly accessible region of an iterator, where the element at index i is given by the expression returned by the function $body(i)$. While this expression is often a tensor access, it could also be a function call, like $f(i) = \sin(\pi i/7)$. Lookups are leaf looplets, and the body is a value, not a looplet.	• $body(i)$: A function which returns an expression representing the value at index i in the current program state.
run(body): The Run looplet represents a constant region of an iterator. Runs are leaf looplets, and the body of a run is a value, not a looplet, similar to a Lookup. Run looplets do not need to store any information about their region because it is specified by the enclosing loop.	• $body$: An expression representing the value within the run in the current program state.
switch(cond, head, tail): The Switch looplet allows us to specialize the body of a looplet based on a condition, evaluated in the embedding context. If the condition is true, we use $head$, otherwise $tail$. Switch has a high lowering priority so we can see the looplets it contains and lower them appropriately. Lowering Switch looplets first also lifts the condition as high as possible into the loop nest.	• $cond$: A function that returns a Boolean value. • $head$: A looplet to execute if the condition is true. • $tail$: A looplet to execute if the condition is false.
thunk(preamble, body, epilogue): The Thunk looplet allows us to cache certain computations in program state. The computations can be used by the Thunk $body$, making Thunks useful for computing and caching the results of expensive computations. The $epilogue$ can be used to clean up any relevant side effects.	• $preamble$: A program that executes before $body$, modifying program state. • $body$: A looplet that can reference variables defined in $preamble$. • $epilogue$: A program that cleans up any relevant side effects of $preamble$ or $body$.
sequence(bodies...): The Sequence looplet represents the concatenation of two or more looplets. The arguments must be $phase$ objects which regions on which each body is defined.	• $bodies...$: One or more phase objects, whose regions must be non-overlapping, covering, and ordered.
phase(ext, body): The Phase object is not a looplet, but instead helpfully couples a sublooplet with the subregion of indices it is defined on in a larger compound looplet.	• ext: An expression representing the absolute range on which the $body$ is defined. • $body$: The looplet describing the sequence within the $range$.
spike(body, tail): The Spike looplet represents a run followed by a single value. Spike can be considered a shorthand for **sequence**(**phase**($i : j-1$, **run**($body$)), **phase**($j : j$, **run**($tail$))). In the Finch compiler, spikes are handled with special care, since they are an opportunity to align the final run to the end of the root loop extent without using any special bounds inference.	• $body$: An expression representing the value within the run. • $tail$: An expression representing the value at the end of the spike.
stepper([seek], next, stride, body): The Stepper looplet represents a variable number of looplets, concatenated. Since our looplets may be skipped over due to conditions or various rewrites, the $seek$ function allows us to fast-forward the state to the start of the root loop extent when it comes time to lower the stepper. **jumper(seek, next, body):** The Jumper looplet is identical to a stepper looplet, but when two jumpers interact, the largest stride between them is taken, and the jumper with the smaller stride is demoted to a stepper within that region. Jumpers allow us to request leader-follower strategies or mutual-lookahead coiteration.	• $seek(j)$: A function that returns a program that advances state to the iteration of the stepper which processes the absolute coordinate j. • $next$: A program that advances the state to the next iteration of the stepper. • $stride$: The absolute endpoint of the current subregion of the stepper. • $body$: The looplet to execute for the current iteration of the stepper.

Table 3. Detailed descriptions of looplet behavior. An example compilation is given later in Figures 15 and 16

2.2 Fiber Trees

Fiber-tree style tensor abstractions have been the subject of extensive study [29, 30, 90]. The underlying idea is to represent a multi-dimensional tensor as a nested vector datastructure, where each level of the nesting corresponds to a dimension of the tensor. Thus, a matrix would be represented as a vector of vectors. This kind of abstraction lends itself to representing sparse

tensors if we vary the type of vector used at each level in a tree. Thus, a sparse matrix might be represented as a dense vector of sparse vectors. The vector of subtensors in this abstraction is referred to as a **fiber**.

Instead of storing the data for each subfiber separately, most sparse tensor formats such as CSR, DCSR, and COO usually store the data for all fibers in a level contiguously. In this way, we can think of a level as a bulk allocator for fibers. Continuing the analogy, we can think of each fiber as being disambiguated by a **position**, or an index into the bulk pool of subfibers. The mapping f from indices to subfibers is thus a mapping from an index and a position in a level to a subposition in a sublevel. Figure 3 shows a simple example of a level as a pool of fibers. When we need to refer to a particular fiber at position p in the level l, we may write $fiber(l, p)$. Note that the formation of fibers from levels is lazy, and the data underlying each fiber is managed entirely by the level, so the level may choose to overlap the storage between different fibers. Thus, the only unique data associated with $fiber(l, p)$ is the position p.

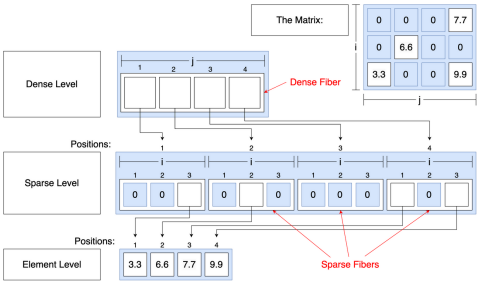


Fig. 3. Levels in the fiber tree representation of a sparse matrix in CSC format, with a dense outer level and a sparse inner level. The element level holds the leaves of the tree.

3 BRIDGING LOOPLETS AND FINCH: THE TENSOR INTERFACE

Tensors use multiple dimensions to organize data with respect to orthogonal concepts. Thus, the Finch language supports multi-dimensional tensors. Unfortunately, the looplet abstraction is best suited towards iterators over a single dimension. Our level abstraction provides a bridge between the single dimensional iterators created from looplets and the multi-dimensional fiber-tree abstractions common to tensor compilers. This bridge must address three challenges. First, while looplets represent an instance of an iterator over a tensor, we may access the same tensor twice with different indices. Thus, the *unfurl* function creates separate looplet nests for each iterator. Next, since Finch programs go beyond just single Einsums, they may read and write to the same data at different times. The *declare*, *freeze*, and *thaw* functions provide machinery to manage transition between these states. Finally, we must be able to write looplet nests that modify tensors, as well as reading them. The *assemble* function manages the allocation of new data in the tensor.

Additionally, prior fiber-tree representations focus on sparsity (where only the nonzero elements are represented) and treat sparse vectors as sets of represented points. Since our fiber-tree representation must handle other kinds of structure, such as diagonal, repeated, or constant values, we must generalize our fiber abstraction to allow arbitrary mappings from indices into a space of subfibers.

In the rest of this section, we discuss how these 5 core functions (*declare*, *freeze*, *thaw*, *unfurl*, and *assemble*) function as part of a life cycle abstraction that defines a level in Finch. These interfaces add to the level abstraction, expanding the types of data that they can express via mapping to looplets and expanding the contexts in which they can be used. We then identify a taxonomy of four key structural properties exhibited in data. We implement several levels in this abstraction that capture all combinations of these structures, including specializations to zero dimensional tensors (scalars) and level structures that support different access patterns.

3.1 Tensor Lifecycle, Declare, Freeze, Thaw, Unfurl

Our simplified view of a level is enabled by our use of looplets to represent the structure within each fiber. In fact, our level interface requires only 5 highly general operations, described below.

The first three of these functions, *declare*, *freeze*, and *thaw*, have to do with managing when tensors can be assumed mutable or immutable. As we use looplets to represent iteration over a tensor, we must restrict the mutability of tensors while we iterate over them. For example, if a tensor declares it has a constant region from $i = 2 : 5$, but some other part of the computation modifies the tensor at $i = 3$, this would result in incorrect behavior. It is much easier to write correct looplet code if we can assume that the tensor is immutable while we are reading from it. Thus, we introduce the notion that a tensor can be in read-only mode or update-only mode. In read-only mode, the tensor may only appear in the right-hand side of assignments. In update-only mode, the tensor may only appear in the left-hand side of an assignment, either being overwritten or incremented by some operator. We can switch between these modes using freeze and thaw functions. The *declare* function is used to allocate a tensor, initialize it to some specified size and value, and leave it in update-only mode.

Description	Arguments
declare(*tns*, *init*, *dims*...) : Returns a program that declares a tensor of size *dims* and an initial value of *init*. Requires the level to be in read-only mode.	• *tns*: The tensor object to declare. • *init*: An expression for the initial value. • *dims*...: Several expressions for the tensor dimensions.
freeze(*tns*) : Returns a program that finalizes the updates in the tensor, and readies the tensor for reading. Requires the tensor to be in update-only mode.	• *tns*: The tensor object to freeze.
thaw(*tns*) : Returns a program that prepares the level to accept updates. Requires the tensor to be in read-only mode.	• *tns*: The tensor object to thaw.
unfurl(*tns*, *ext*, *mode*) : Returns a looplet that iterates over subfibers within the fiber at position *pos* in the level *lvl* over the extent *ext*. When *mode* = **read**, returns a looplet nest over the values in the read-only fiber. When *mode* = **update**, returns a looplet nest over mutable subfibers in the update-only fiber. We call *unfurl* directly before iterating over the corresponding loop, so it has access to any state variables introduced by freezing or thawing the tensor.	• *lvl*: The level object of the tensor to unfurl. • *pos*: An expression representing the position of the fiber to unfurl. • *ext*: An expression representing the range to unfurl over. • *mode*: An enum representing whether to unfurl in read-only or update-only mode.
unwrap(*tns*, *mode*, [*op*], [*rhs*]) : Returns code to read or update the scalar value of a scalar or leaf node *tns*, using *op* and *rhs* in the case of update. Parent fibers may ask their children to use this function to set a dirty bit in *tns*, indicating a non-fill value has been written and that the child fiber needs to be stored.	• *tns*: The tensor object to increment, possibly a fiber. • *mode*: An enum representing whether to unwrap in read-only or update-only mode. • *op*: An expression representing the operation to apply to the scalar value. • *rhs*: An expression representing the second argument to *op*.
assemble(*lvl*, pos_{start}, pos_{stop}) : Returns a program that allocates subfibers in the level from positions pos_{start} to pos_{stop}. In looplet nests which modify the output, this function is often called to construct the output tensor. For example, as a new nonzero is discovered, we might call *assemble* to obtain a location in memory to which we can write the nonzero.	• *lvl*: The level object in which subfibers are allocated. • pos_{start}: The first subfiber position to assemble. • pos_{stop}: The last subfiber position to assemble.

Table 4. The five functions that define a level.

The *unfurl* function is used to manage iteration over a subfiber. When it comes time to iterate over a tensor, be in on the left or right hand side of an assignment, we call *unfurl* to return a looplet nest that describes the hierarchical structure of the outermost dimension of the tensor. We call *unfurl* directly before iterating over the corresponding loop, so it has access to any state variables introduced by freezing or thawing the tensor. Looplets were chosen for this purpose as a symbolic engine to ensure certain simplifications take place, but another symbolic system could have been used (e.g. polyhedral[106] or e-graph search [83]). We chose looplets because they reliably process structured iterators, predictably eliminating zero regions, using faster lookups when available, and utilizing repeated work.

Our view of a level as a fiber allocator implies an allocation function $assemble(tns, pos_{start} : pos_{stop})$, which allocates fibers at positions $pos_{start} : pos_{stop}$ in the level. We don't specify a deallocation function, instead relying on initialization to reset the fiber if it needs to be reused. While all of the previous functions are used to manage the lifecycle and iteration over a general tensor, *assemble* is quite specific to the level abstraction, and the notion of positions within sublevels. Note: it was an intentional choice to hold the parent level responsible for managing the data of the sublevels, which positions they allocate, etc. This allows the parent level to reuse allocation logic from internal index datastructures. For example, a sparse level might use a list of indices to store which nonzeros are present, and when it comes time to resize that list, it could also call *assemble* to resize the sublevel, reducing the number of branches in the code. The *assemble* function lends itself particularly to a "vector doubling" allocation approach, which we have found to be effective and flexible when managing the allocation of sparse left hand sides. This benefit is made clear in our case studies, where prior systems like TACO do not support all possible loop orderings and format combinations for sparse matrix multiply because they do not have a flexible enough allocation strategy, instead using a two-phase approach which requires computing a complicated closed-form kernel to iterate over the data twice to determine the number of required output nonzeros.

3.2 The 4 Key Structures

In the Finch programming model, the programmer relies on the Finch compiler to specialize to the sequential properties of the data. In our experience, the main benefits of specializing to structure come from the following properties of the data:

- **Sparsity** Sparse data is data that is mostly zero, or some other fill value. When we specialize on this data, we can use annihilation ($x * 0 = 0$), identity ($x * 1 = 1$), or other constant propagation properties ($ifelse(false, x, y) = y$) to simplify the computation and avoid redundant work.
- **Blocks** Blocked data is a subset of sparse data where the nonzeros are clustered and occur adjacent to one another. This provides us with two opportunities: We can avoid storing the locations of the nonzeros individually, and we can use more efficient randomly accessible iterators within the block. [7, 56, 98].
- **Runs** Runs of repeated values may occur in dense or sparse code, cutting down on storage and allowing us to use integration rules such as `for i = 1:n; s += x end` ⟶ `s += n * x` or code motion to lift operations out of loops [7, 36].
- **Singular** When we have only one non-fill region in sparse data, we can avoid a loop entirely and reduce the complexity of iteration [7, 44].

Sparse	Blocked	Runs	Singular	Corresponding Format
				Dense
			✓	n/a
		✓		RunList
		✓	✓	n/a
	✓			n/a
	✓		✓	n/a
	✓	✓		n/a
	✓	✓	✓	n/a
✓				SparseList
✓			✓	SparsePinpoint
✓		✓		SparseRunList
✓		✓	✓	SparseInterval
✓	✓			SparseBlockList
✓	✓		✓	SparseBand
✓	✓	✓		n/a
✓	✓	✓	✓	n/a

Fig. 4. All combinations of our 4 structural properties and the corresponding formats we have chosen to represent them. Not all combinations are relevant. Note that blocks and runs need not be considered together because we must store a run length for each run, so there isn't a significant storage benefit to combining them. Blocks and singletons only make sense in the context of sparsity.

In the following section, we consider a set of concrete implementations of levels that expose all combinations of these structures, paying some attention to a few important special cases: random access, scalars, and leaf levels. We summarize the structures in Table 5 and Table 4.

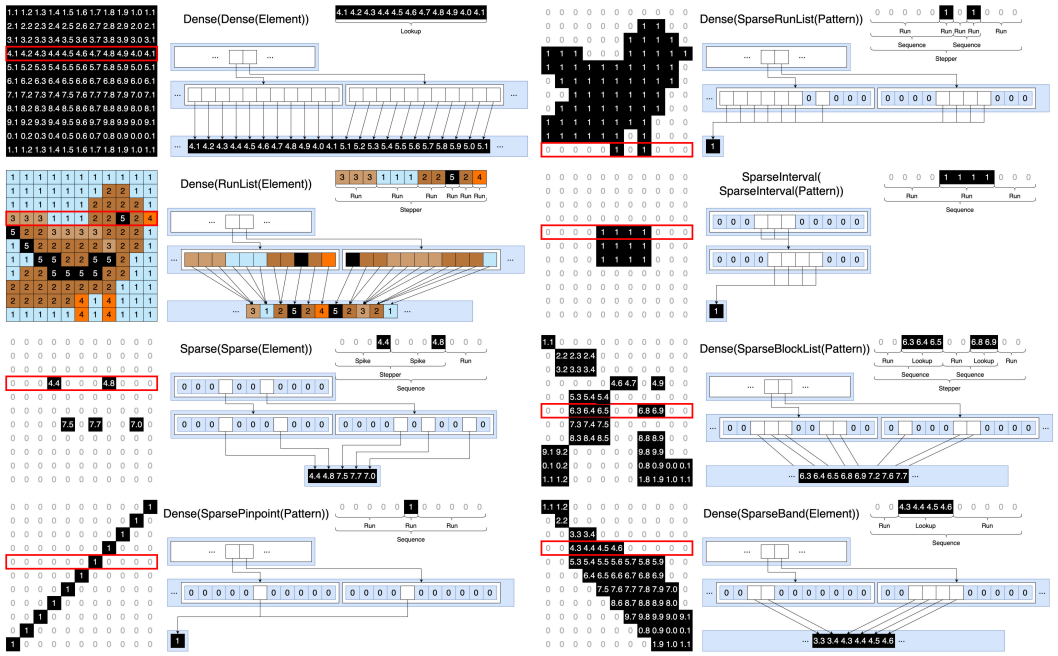


Fig. 5. Several examples of matrix structures represented using the level structures identified in Table 4. Comparing this figure to [7, Figure 3], we see that a level-by-level structural decomposition is diagrammed together with the looplets.

3.3 Implementations of Structures

3.3.1 Sequentially Constructed Levels. We consider all combinations of these structural properties in Table 4, resulting in 8 key level formats that correspond to the 8 resulting situations. While it is impossible to write code which precisely addresses every possible structure, our level formats can be combined to express a wide variety of hierarchical structures to a sufficient granularity that we can generate code which utilizes the four properties. For example, though banded tensors are a superset of ragged tensors, representing a ragged tensor using a banded format does not add much overhead. The structures we consider are exhaustive in the sense that they address all combinations of sparsity, blocks, runs, and singletons in each level. We can represent a wide variety of hierarchical tensor structures by combining these level structures in a tree, as shown in Figure 5.

3.3.2 Non-sequentially Constructed Levels. To reduce the implementation burden and improve efficiency in the common case, the levels we described in the previous section only support bulk, sequential construction of formats. However, when users want to be able to write out of order (which is a common requirement arising from loop order or from the problem itself, it occurs in our SpGEMM algorithms and our histogram example in the evaluation section), we must use more complicated datastructures like hash tables and trees to support the random writes so that in-order levels can be constructed later. Because these datastructures are more complex and have a higher implementation burden and performance overhead, we only support random access construction of sparse or dense structures. We can use these two more general structures as intermediates to convert to our more specialized structures later.

3.3.3 Scalars. Because leaf levels are geared towards representing multiple leaves, we also introduce a much simpler Scalar format to represent 0-dimensional tensors. Scalars don't have as much structure because they only concern one value. However, we allow the programmer to declare that

Sequentially Constructed Levels
Dense: The dense format is the simplest format, mapping $fiber(l, p)[i] \rightarrow fiber(l.lvl, p \times l.shape + i)$. This format is used to store dense data and is often a convenient format for the root level of a tensor. Due to its simplicity, freezing and thawing the level are no-ops.
RunList: Used to represent runs of repeated values, storing two vectors, $right$ and ptr, with q^{th} run in the p^{th} subfiber starting and ending at $right[ptr[p] + q]$ and $right[ptr[p] + q + 1] - 1$, respectively. A challenge arises for this level: it is difficult to merge duplicate runs. Such a scenario might arise when merging runs of subfibers of length 3, representing colors in an image. Ideally, we would be able to detect duplicate subfibers and merge them on the fly, but we cannot determine which subfibers are equal because we cannot read them while the sublevel is in update-only mode. Instead, we merge the duplicates during the freeze phase. We $freeze$ the sublevel, $declare$ a separate sublevel buf as a buffer to store the deduplicated subfibers, and then compare each of the subfibers in the main level, copying the deduplicated subfibers into the buffer.
SparseList: The simplest sparse format, used to construct popular formats like CSR, CSC, DCSR, DCSC, and CSF. It stores two vectors, idx and ptr, such that $idx[ptr[p] + q]$ is the index of the q^{th} nonzero in the subfiber at position p.
SparsePinpoint: Similar to SparseList, but only one nonzero in each subfiber, eliminating the need for the ptr field. It stores a vector idx, such that $idx[p]$ is the nonzero index in the subfiber at position p.
SparseRunList: Similar to RunList level, but because runs are sparse, we must also store the start of each run. It stores three vectors $left$, $right$, and ptr, such that the q^{th} run in the p^{th} subfiber begins and ends at $left[ptr[p] + q]$ and $right[ptr[p] + q]$, respectively. Like RunList, it also stores a duplicate sublevel, buf, for deduplication.
SparseInterval: Similar to SparseRunList, but only stores one run per subfiber, eliminating the need for the ptr field. This level does not deduplicate as it cannot store intermediate results with more than one run. It stores two vectors, such that the run in subfiber p begins and ends at $left[p]$ and $right[p]$ respectively.
SparseBlockList: Used to represent blocked data. It stores three vectors, idx, ptr, and ofs, such that $ofs[ptr[p]+q] : ofs[ptr[p]+q+1]-1$ are the subpositions of block q ending at index $idx[ptr[p] + q]$ in the subfiber at position p.
SparseBand: Similar to SparseBlockList, but stores only one block per subfiber, eliminating the need for the ptr field. It stores two vectors idx and ofs, such that $ofs[p] : ofs[p + 1] - 1$ are the subpositions of the block ending at $idx[p]$ in subfiber p.
Nonsequentially Constructed Levels
SparseHash: The sparse hash format uses a hash table to store the locations of nonzeros, and sorts the unique indices for iteration during the freeze phase. This allows for efficient random access, but not incremental construction, as the freeze phase runs in time proportional to the number of nonzeros in the entire level. It stores two vectors, idx and ptr, such that $idx[ptr[p] + q]$ is the index of the q^{th} nonzero in the subfiber at position p. Also stores a hash table tbl for construction and random access in the level.
SparseBytemap The SparseBytemap format uses a bytemap to store which locations have been written to. Unlike the SparseHash format, the bytemap assembles the entire space of possible subfibers. This accelerates random access in the format, but requires a high memory overhead. Because we don't want to reallocate all of the memory in each iteration, the declaration of this format instead re-assembles only the dirty locations in the tensor. This format is analogous to the default workspace format used by TACO. It stores two vectors, idx and ptr, such that $idx[ptr[p] + q]$ is the index of the q^{th} nonzero in the subfiber at position p. These vectors are used to collect dirty locations. It also stores tbl, a dense array of Booleans such that $tbl[shape * p + i]$ is true when there is a nonzero at index i in the subfiber at position p.
Leaf Levels
Element: The element level uses an array val to store a value for each position p. The zero (fill) value is configurable.
Pattern: The pattern level statically represents a leaf level with a fill value of $false$ and whose stored values are all $true$.
Scalars
Scalar: A dense scalar that, unlike a variable, supports reduction.
SparseScalar: A scalar with a dirty bit which specializes on the fill value when it occurs.
EarlyBreakScalar: A scalar which triggers early breaks in reductions whenever an annihilator is encountered.

Table 5. The main level formats supported by Finch. Note that all non-leaf levels store a the dimension of the subfibers and a child level. Since we must be able to handle the case where a sublevel is not stored because a parent level is sparse, all of Finch's sparse formats use a dirty bit during writing to determine whether the sublevel has been modified from it's default fill value and thus, whether it needs to be stored.

a scalar might be sparse, or that it might be used in a reduction which can be exited early. Through these structures, scalars can also affect other tensors in crucial ways.

When a scalar is sparse, this means that it might be equal to the fill value and the user has requested for the compiler to simplify subsequent computations accordingly. Constant propagation through tensors is known to be a complex compiler pass [72]. We provide sparse scalars as an alternative, which allow for similar semantics by specializing reads for the possible fill value.

ShortCircuitScalars trigger stepper looplets to re-specialize the loop whenever a reduction into the scalar hits an annihilator, removing the reduction from the specialized case since it has hit the annihilator value and can no longer change. We don't need to re-specialize other looplets because the stepper is the only one which repeats a non-constant number of times.

We also provide ShortCircuitScalars, which signal that we should check for an opportunity to early break out of a reduction loop when we hit an annihilator value. For example, Figure 6 computes the product of the elements in a vector, exiting the loop when one of them is zero.

```
p = ShortCircuitScalar{0}()
@finch begin
    p .= 0
    for j=_
        p[] *= A[j]
```

Fig. 6. Using a ShortCircuitScalar to find the product of values in A.

We represent early break as a structural property rather than a program node because it allows us to represent the tail of a loop where one scalar has hit an annihilator but another scalar hasn't. The effects of a **break** statement would affect the value of all other statements in the loop, and violate some of the dataflow assumptions lifecycle constraints would otherwise permit. Representing breaks as structural properties allows us to more elegantly compose break statements with other structures in the language. To our knowledge, sparse and ShortCircuitScalars are novel contributions of this work; other systems don't include them, limiting the impact of sparsity.

3.3.4 Leaf Levels. The leaf level stores the actual entries of the tensor. In most cases, it is sufficient to store each entry at a separate position in a vector. This is accomplished by the **ElementLevel**. However, when all of the values are the same, an additional optimization can be made by storing the identical value only once. In this work, we introduce the concept of a **PatternLevel** to handle this binary case. The PatternLevel has a fill value of $false$, and returning $true$ for all "stored" values. The PatternLevel allows us to easily represent unweighted graphs or other Boolean matrices.

4 THE FINCH LANGUAGE

```
EXPR := LITERAL|VALUE|INDEX|VARIABLE|EXTENT|CALL|ACCESSS
STMT := ASSIGN|LOOP|DEFINE|SIEVE|BLOCK|DECLARE|FREEZE|THAW

 DECLARE := TENSOR .= EXPR(EXPR...)   #V is the set of all values
  FREEZE := @freeze(TENSOR)           #S is the set of all Symbols
    THAW := @thaw(TENSOR)             #T is the set of all types
  TENSOR := TENSORNAME :: WRAPPER(TENSOR, EXPR...)
  ASSIGN := ACCESS <<EXPR>>= EXPR      TENSORNAME := S
    LOOP := for INDEX = EXPR           LITERAL := V
              STMT                       VALUE := S::T
            end                          WRAPPER := S
  DEFINE := let VARIABLE = EXPR          INDEX := S
              STMT                     VARIABLE := S
            end                          EXTENT := EXPR : EXPR
   SIEVE := if EXPR                        CALL := EXPR(EXPR...)
              STMT                     ACCESS := TENSOR[EXPR...]
            end                          MODE := @mode(TENSOR)
   BLOCK := begin
              STMT...
            end
```

(a) The syntax of the Finch language. Compare this grammar to the Concrete Index Notation of TACO [59, Figure 3], noting the addition of multiple left-hand sides through code blocks, access with arbitrary expressions, and explicit declaration, as well as freeze and thaw.

$$[\![\mathbf{loop}(i,\mathbf{extent}(a,b),\mathbf{block})]\!]^F = \cup^F_{iv \in \mathbb{Z} \cap [[\![a]\!]^F, [\![b]\!]^F]} [\![\mathbf{block}]\!]^{F, i \mapsto iv}$$

$$[\![\mathbf{access}(tensor,exprs...)]\!]^F = [\![tensor]\!]^F([\![exprs]\!]^F...)$$

$$[\![tensorname]\!]^F = F(tensorname)$$

$$[\![wrapper(tensor,exprs)]\!]^F = ([\![wrapper]\!]^W(exprs))([\![tensor]\!]^F)$$

$$[\![\mathbf{block}(stmt_1,stmts...)]\!]^F = [\![stmt_1]\!]^F \cup^F [\![\mathbf{block}(stmts...)]\!]^F$$

$$[\![\mathbf{block}()]\!]^F = \{\}$$

$$[\![\mathbf{sieve}(expr,stmt)]\!]^F = \begin{cases} [\![stmt]\!]^F & [\![expr]\!]^F \\ \{\} \end{cases}$$

$$[\![\mathbf{declare}(var,expr,stmt)]\!]^F = [\![stmt]\!]^{F, var \mapsto [\![expr]\!]^F}$$

$$[\![\mathbf{assign}(\mathbf{access}(tensor,idxExpr),op,expr)]\!]^F = F \cup^F \{tensor \mapsto [\![tensor]\!]^F \cup \{[\![idxExprs]\!]^F... \mapsto [\![op]\!]^F([\![tensor]\!]^F([\![idxExprs]\!]^F...),[\![expr]\!]^F)\}\}$$

(b) Semantics of Finch: The semantic domains are F, an assignment of tensor names to functions ($\mathbb{Z}^N \to V$) as well as W, an assignment of wrappers to functions with type $V^M \mapsto ((\mathbb{Z}^N \mapsto V) \mapsto \mathbb{Z}^{N'} \mapsto V)$, representing transformations of tensors. The dimension $a : b$ of an index i or a declaration is computed via the rules laid out in Section 4.1.

Fig. 7. Syntax and Semantics for Finch

The syntax of Finch is displayed in Figure 7a, and a denotational semantics is displayed in Figure 7b. The Finch language mirrors most imperative languages such as C with for-loops and control flow. Notable statements that have been added to the language include **for**, **let**, blocks of code with **if**, wrappers of tensors, and the lifecycle functions that let us declare, freeze, and thaw tensors.

The denotational semantics of our language concern large dense iteration spaces, but the implementation eliminates many of these unnecessary iterations through aggressive optimizations,

carefully using life cycles, dimensions, sparsity via looplets, and control flow as a form of sparsity. Section 5 details the specifics of how we compile our syntax to efficient code over structured data.

Our expressions support a wide variety of scalar operations on literals, indices, extents, wrappers, and calls to externally defined functions. Wrappers are static higher order functions on tensors that serve to implement complex indexing logic such as $i + j$ or $i <= j$; an initial pass in the compiler converts indexing logic to a wrapper function when possible. Since the wrappers are rather simpler higher order functions, we can implement them as transformations on looplets or other properties of the tensor format, which will mean careful implementations of wrapper will allow a more efficient, lazy implementation of complex tensor accesses as opposed to naive look ups or naively rebuilding a tensor at each iteration. For examples of wrappers, see Table 6. Finally, as detailed in the previous section, tensors are defined externally via an interface that supports the *declare*, *freeze*, *thaw*, and *unfurl* functions. The first three are supported directly in the syntax whereas the fourth will be introduced through evaluation of loops and accesses, in the next section. We do not intend the user to insert freeze or thaw manually, but we include them in the language because it allows us to handle tensor lifecycles with a separate, simpler, compiler pass, rather than all at once. Tensors can only change between read and write mode in the scope in which they were defined, so we can insert freeze/thaw automatically by checking whether the tensor is being read or written to in each child scope. We error if a tensor appears on both the left hand and right hand sides within the same child scope. This algorithm is described later in Section5.

Our syntax is highly permissive: by allowing blocks of code with multiple statements, we implicitly support many features gained through complicated scheduling commands in other frameworks, such as multiple outputs, masking to avoid work, temporary tensors, and arbitrary loop fusion and nesting. These features are seen most prominently in our implementation of Gustavson's algorithm for sparse-sparse matrix multiply, which simply writes to a temporary tensor in an inner loop and then reuses it; or in our breadth-first search, which uses an `if` statement to avoid operating on vertices outside the frontier. The only restriction we impose on our syntax is that it must respect tensor life cycles. As discussed in Section 3.3, our language does not include a `break` statement. We instead represent breaks as a structural property, so that they can more elegantly compose with other structures in the language.

4.1 Dimensionalization Rules

Looplets typically require the dimension of the loop extent to match the dimensions of the tensor. However, it is cumbersome to write the dimensions in loop programs, and most tensor compilers have a means of specifying the dimensions automatically. In many pure Einsum languages like TACO, determining dimensions is not needed because any tensor dimensions that share an index are assumed to be the same [60]. Other languages, such as Halide, perform bounds inference where known bounds are symbolically propagated to fill in unknown bounds, often from output/input sizes to intermediates via some approximation such as interval analysis or polyhedral methods [47, 78]. We refer to the process of discovering suitable dimensions as **dimensionalization**.

Loop bounds in Finch are computed automatically via a few simple rules. There are currently two kinds of dimensions in Finch: `_` represents a dimensionless quantity, and `a:b` represents an integer dimension. Dimensions can be joined with the `meet` operation, which returns the dimension that is not `_` or else asserts that the two extents match.

- The dimension of an index is defined as the `meet` of the loop bound and the tensor dimension corresponding to any right-hand-side accesses with that index.
- The n^{th} dimension of a tensor declaration is defined as the `meet` of all index dimensions in the n^{th} mode of left-hand-side accesses to that tensor, from its declaration to its first read.

- The dimension of `i + c`, where `c` is a constant, is the dimension of `i` shifted by `c`.
- The dimension of `~(x)`, or any other unrecognized function, is `_`.
- More rules may be added as Finch is extended to recognize more indexing syntax.

5 THE FINCH COMPILER

The Finch compiler takes a Finch program together with a program state defining the formats of tensors, and produces efficient structure aware code. The compiler operates in several stages. The first stages normalize the program to make it easier to process. The final stages lower a normalized program recursively, one loop at a time. For each loop, all tensors that are indexed by the loop index are transformed into looplets based on their structure, and these looplets are lowered to executable code. The overall flow is summarized in Figure 8.

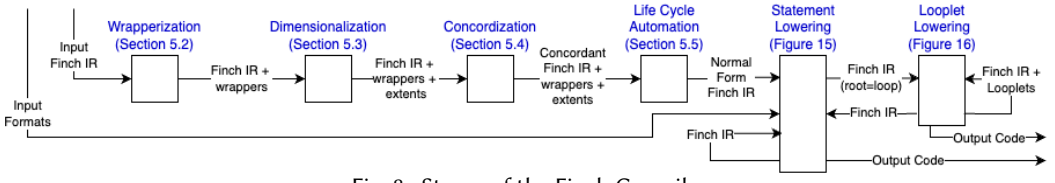


Fig. 8. Stages of the Finch Compiler.

5.1 Finch Normal Form

Our core recursive lowering compiler described in Figure 13 and Figure 14 is designed to handle a particular class of programs we refer to as **Finch Normal Form**. This section defines the properties of Finch Normal Form. Later sections will describe how to normalize all Finch programs which are well-defined under the semantics in Figure 7b.

The properties of such a program are as follows:

- **Access with Indices:** Though Finch allows general expressions (including affine expressions and general function calls) in an access (i.e. `A[i + j]` or `A[I[i]]`), the normal form restricts to allow only indices in accesses (i.e. `A[i]`), rather than more general expressions.
- **Evaluable Dimensions:** Loop dimensions and declaration dimensions must be evaluable at the time we compile them, so we restrict the normal form to programs whose loop dimensions and declaration dimensions are extents with limits defined in the scope of the corresponding loop or declaration statement.
- **Concordant:** Finch is column-major by default to match Fortran[10] and Julia[19]. A Finch program is **concordant** when the order of indices in each access match the order in which loops are nested around it. For example, `for j = _; for i = _; s[] += A[i, j] end end` is concordant but `for i = _; for j = _; s[] += A[i, j] end end` is not.
- **Lifecycle Constraints:** Tensors in read mode may appear on the right hand side only. Tensors in update mode may appear on the left hand side only. To make it easier to analyze lifecycle constraints statically, we restrict tensors to only change modes in the same scopes in which they were defined.

The subsequent sections will explain how programs that violate each of these constraints can be rewritten to programs that satisfy them and thus how we can support such a wide variety of programs. For example, we can write nonconcordant programs like `for i = _; for j = _; s[] += A[i, j] end end` by inserting a loop to randomly access `A`.

5.2 Wrapperization

Many fancy operations on indices can be resolved by introducing equivalent **wrapper tensors** which modify the behavior of the tensors they wrap, or by introducing **mask tensors** which

replace index expressions like `i <= j` with their equivalent masks (in this case, a triangular mask tensor). Wrappers and masks are summarized in Table 6.

```
for i=_, j=_                    for i=_, j=_                    for i = 1:n
  if i <= j                ⟶     if UpTriMask()[i, j]     ⟶     for j = 1:i
    s[] += A[i - 1, j]              s[] += OffsetTensor(A, (-1, 0))[i, j]          s[] += A.val[(i - 1) + j * n]
```

Fig. 9. Wrapperization. While `i <= j` is only an expression, `UpTriMask()[i, j]` can use looplets to end each loop over `j` at `i`.

OffsetTensor shifts tensors such that `offset(tns, delta...)[i...] == tns[i + delta...]`. The shifting is achieved by modifying the ranges returned by the looplets in the wrapped tensor. `A[i..., j + c, k...] -> OffsetTensor(A, (0..., c, 0...))[i..., j, k...]`	
ToeplitzTensor adds a dimension that shifts another dimension of the original tensor. The added dimensions are produced during a call to `Unfurl`, when a lookup looplet is emitted for the first dimension. `A[i_1, ..., i_n, j + k, l...] -> ToeplitzTensor(A, n)[i_1, ..., i_n, j, k, l...]`	
PermissiveTensor allows for out-of-bounds access or padding. PermissiveTensor returns a dimensionless value for any permissive indices. PermissiveTensor returns `missing` as the out-of-bounds value, where the `coalesce` function can be used to return the first nonmissing value. `A[i..., ~j, k...] -> PermissiveTensor(A, (false..., true, false...))[i..., j, k...]`	
ProtocolizedTensor allows for advanced iteration protocols. The ProtocolizedTensor selects between several different implementations of `unfurl` that a level may support. `A[i..., p(j), k...] -> ProtocolizedTensor(A, (nothing..., p, nothing...))[i..., j, k...]` Finch recognizes several protocols: • The `follow` protocol indicates we should ignore the structure and randomly access each element. • The `walk` protocol declares that the structure of the iterator should be used in the computation. • The `gallop` protocol declares that the structure of a tensor should lead an iteration and we should specialize to that structure with a higher priority than others. A galloping protocol over two SparseList levels produces a mutual-binary-search merge algorithm popularized in the case of worst-case-optimal join queries [13, 74, 95].	
SwizzleTensor is a lazily transposed tensor that changes the interpretation of the order of modes in the tensor. Unlike other wrappers, a SwizzleTensor is compiled during the wrapperization pass rather than introduced by it. `swizzle(A, perm)[idx...] -> A[idx[perm]...]`	

UpTriMask is a mask tensor that represents Boolean triangular matrices. `UpTriMask()[i, j]` is true when `i <= j`.
It is introduced via several rewrite rules, such as:
`i < j -> UpTriMask()[i, j - 1]`
`i > j -> !UpTriMask()[i, j - 1]`
`i <= j -> UpTriMask()[i, j]`
`i >= j -> !UpTriMask()[i, j]`
When `i` would be bound at a higher loop depth than `j`, care is taken to reverse the loop order and emit the mask in column-major order.
	`unfurl(UpTriMask(), ext, reader) =`
	`  Lookup(body(j) = UpTriMaskCol(j))`
	`unfurl(UpTriMaskCol(j), ext, reader) =`
	`  Sequence(`
	`  Phase(`
	`  stop = j,`
	`  body = Run(true)),`
	`  Phase(`
	`  body = Run(false)))`

DiagMask is a mask tensor that represents Boolean diagonal matrices. `DiagMask()[i, j]` is true when `i == j`.
It is introduced via several rewrite rules, such as:
`i == j -> DiagMask()[i, j]`
`i != j -> !DiagMask()[i, j]`
When `i` would be bound at a higher loop depth than `j`, care is taken to reverse the loop order and emit the mask in column-major order.
`unfurl(DiagMask(), ext, reader) =`
`  Lookup(body(j) = DiagMaskCol(j))`
	`unfurl(DiagMaskCol(j), ext, reader) =`
	`  Sequence(`
	`  Phase(`
	`  stop = j - 1,`
	`  body = Run(false)),`
	`  Phase(`
	`  stop = j,`
	`  body = Run(true)),`
	`  Phase(`
	`  body = Run(false)))`

Table 6. Wrapper tensors

More formally, a **wrapper tensor** is any tensor that wraps a tensor variable in an access, and can overload the behavior of *unfurl*, *unwrap*, and *size*, as well as modify the ranges declared by any looplets the wrapper contains. For example, the `offset` wrapper tensor shifts looplets to shift the tensor with respect to the loop index. Wrappers are implemented either through a program rewrite during the wrapperization procedure, or by overloading format and looplet APIs 14 with some minor modifications. For example, OffsetTensor shifts the declared ranges of contained looplets.

A **mask tensor** is simply a Boolean Finch tensor with implicit structure that uses a predefined looplet nest, rather than the level abstraction. For example, the `UpTriMask` tensor uses looplets to represent the structure of a Boolean upper triangular matrix. Mask tensors are implemented using static looplets that are constructed during the unfurl step. Mask tensors allow us to lift computations with masks to the level of the loop, without modifying the loop directly.

5.3 Dimensionalization

In Section 4.1, we described a simple set of rules to calculate dimensions. In Finch, these rules are implemented through a straightforward dimensionalization algorithm which operates on loops and declaration statements (output tensors). Finch determines the dimension of a loop index i from all of the tensors using i in an access, as well as the bounds in the loop itself, and operates similarly for declarations.

```
#A is 3 x 4
#B is 4 x 5
C .= 0
for i = 1:3
  for j = _
    for k = _
      C[i, j] += A[i, k] * B[k, j]
```

↓

```
C .= 0
for i = 1:3
  for j = 1:5
    for k = 1:4
      C[i, j] += A[i, k] * B[k, j]
```

Fig. 10. Dimensionalization

We can compute these dimensions in a single pass over the program. When we reach a **read** access, the tensor must be dimensionalized and we use those dimensions to compute the loop index dimension. When we reach an **update** access, we make note of the indices for later. Because the **freeze** statement must occur outside of any loops which access a tensor, we can assume that those stored indices are dimensionalized and use them to compute the dimensions of the corresponding tensor declaration.

For example, in Figure 10, the second dimension of A must match the first dimension of B. Also, the first dimension of A must match the i loop dimension, 1:3. Finch will resize declared tensors to match indices used in writes, so C is resized to (1:3, 1:5). If no dimensions are specified elsewhere, then Finch will use the dimension of the declared tensor. Dimensionalization occurs after wrapper tensors are de-sugared, so wrapper tensors can be used to pass dimensions through more complex index expressions. The user can exempt an index from dimensionalization by wrapping it in ~ to produce a "PermissiveTensor" (e.g. `A[~i]`) which has dimension `_`.

5.4 Concordization

After wrapperization, we run a pass over the code to make the program concordant. We make expressions concordant by inserting single-iteration loops. Examples are given in Figure 11. The algorithm works by applying the following rewrite rule in any scope where the expression x is bound:

```
for i = _                    for i = _
    ...                          ...
    ... A[x, i] ...     ⟶       for j = x:x
    ...                            ... A[j, i] ...
```

The variable x might be bound by a for-loop or an earlier definition, or it may be a constant.

```
for i = _                          for i = _
  for j = _                          A[I[i]] += 1
    s[] += A[i, j]
```

↓ ↓

```
for i = _                          for i = _
  for j = _                          for j = I[i]:I[i]
    for k = i:i                        A[j] += 1
      s[] += A[k, j]
```

Fig. 11. Examples of concordization, transforming accesses to normal column major.

5.5 Life Cycle Automation

The last normalization pass inserts the `@freeze` or `@thaw` macros automatically. Tensors are only allowed to change mode within the scope in which they were declared. If they have not been inserted already, this pass automatically inserts these statements in the program, easing the programmer's burden and bridging between structured and dense languages.

```
y .= 0                                 y .= 0
for i = _                              for i = _
    y[i] = x[i] + 1                        y[i] = x[i] + 1
for i = _                              @thaw(x)
    x[i] += 1                    ⟶    for i = _
    y[i] += 1                              x[i] += 1
for i = _                                  y[i] += 1
    x[i] += y[i]                       @freeze(y)
                                       for i = _
                                           x[i] += y[i]
                                       @freeze(x)
```

Fig. 12. Life cycle automation.

The pass walks the program and tracks the current mode of each tensor, depending on whether the tensor is read or updated in each statement within the tensor's declared scope block.

5.6 Recursive Lowering

Finally, after normalization, the program is lowered recursively, node by node. This phase is presented as a staged execution of a small step operational semantics for Finch Normal Norm programs. Figure 13 evolves Finch control flow towards loops. Figure 14 lowers loops with looplets.

This stage of the compiler carefully intermixes our control flow and tensors into looplets so the combination can be successfully symbolic simplified together. The crux of this is that loops enter into looplets via the *Unfurl* rule. Unfurl is defined in Section 3.3. In this system, repeated structures and constants are slowly uncovered as accesses are lowered in various points in the program (e.g. *Run* and *Switch*, respectively). In this process, we are able to use rewrite rules in *Simplify* to eliminate cases, unnecessary iterations, and so forth based on the information provided via looplets and via the control flow (loops, sieve, definitions). Because the systems above this reliably transforms complex index accesses and control flow to control flow and wrappers via concordization and wrapperization, this stage of the compiler can use looplets to simplify the combination of tensor structures and control flow to eliminate unneeded work. This also crucially relies on the tensor life cycles as otherwise arbitrary mixes of reads and writes would disrupt the simplifications of looplets, which is why we include life cycles in these semantics.

$$\frac{\langle val, (e,t,d)\rangle \to val' \qquad var \notin d}{\langle \mathbf{define}(var, val, body), (e,t,d)\rangle \to \langle body, (e[var \mapsto val'], t, \{\})\rangle}\textit{Define}$$

$$\frac{}{\langle \mathbf{literal}(val), (e,t,d)\rangle \to val}\textit{Literal}$$

$$\frac{}{\langle \mathbf{variable}(name), (e,t,d)\rangle \to e(\mathbf{variable}(name))}\textit{Variable}$$

$$\frac{\langle args_i, (e,t)\rangle \Rightarrow vals_i \qquad \langle f, (e,t)\rangle \Rightarrow g}{\langle \mathbf{call}(f, args...), (e,t)\rangle \to \langle\!\langle g(vals...), t\rangle\!\rangle}\textit{Call}$$

$$\frac{}{\langle \mathbf{index}(name), (e,t,d)\rangle \to e(\mathbf{index}(name))}\textit{Index}$$

$$\frac{\langle node, algebra\rangle \to node'}{\langle E[node], s\rangle \to \langle E[node'], s\rangle}\textit{Simplify}$$

$$\frac{\langle body, s\rangle \to s'}{\langle \mathbf{block}(body, tail...), s\rangle \to \langle \mathbf{block}(tail...), s'\rangle}\textit{Block}$$

$$\frac{\langle cond, (e,t,d)\rangle \Rightarrow true}{\langle \mathbf{sieve}(cond, body), (e,t,d)\rangle \to \langle body, (e,t,\{\})\rangle}\textit{SieveTrue}$$

$$\frac{e(tns) \mapsto tns' \qquad e(\mathbf{mode}(tns)) \mapsto \mathbf{read} \qquad \langle\!\langle unwrap(tns', \mathbf{read}), t\rangle\!\rangle \to tns''}{\langle E[\mathbf{access}(tns)], s\rangle \to \langle E[tns'']\rangle}\textit{Access}$$

$$\frac{\langle cond, s\rangle \Rightarrow false}{\langle \mathbf{sieve}(cond, body), s\rangle \to s}\textit{SieveFalse}$$

$$\frac{e(tns) = tns' \qquad \langle op, (e,t,d)\rangle \to op' \qquad \langle rhs, (e,t,d)\rangle \to rhs' \qquad e(\mathbf{mode}(tns)) = \mathbf{update} \qquad \langle\!\langle unwrap(tns', \mathbf{update}, op', rhs'), t\rangle\!\rangle \to t'}{\langle E[\mathbf{assign}(\mathbf{access}(tns), op, rhs)], (e,t,d)\rangle \to (e, t', d)}\textit{Assign}$$

$$\frac{}{\langle \mathbf{value}(ex, type), (e,t,d)\rangle \Rightarrow \langle\!\langle ex, t\rangle\!\rangle}\textit{Value}$$

$$\frac{s = (e,t,d) \qquad tns \notin d \qquad e(tns) = tns' \qquad \langle init, s\rangle \Rightarrow init' \qquad \forall i \langle init, dims_i\rangle \Rightarrow dims_i' \qquad \langle\!\langle declare(tns', init', dims'...), t\rangle\!\rangle \to t'}{\langle \mathbf{declare}(tns, init, dims), s\rangle \to (e[\mathbf{mode}(tns) \mapsto \mathbf{update}], t', d \cup \{tns\})}\textit{Declare}$$

$$\frac{s = (e,t,d) \qquad e(\mathbf{mode}(tns)) = \mathbf{update} \qquad tns \in d \qquad e(tns) = tns'}{\langle \mathbf{freeze}(tns), s\rangle \to (e[\mathbf{mode}(tns) \mapsto \mathbf{read}], \langle\!\langle freeze(tns'), t\rangle\!\rangle, d)}\textit{Freeze}$$

$$\frac{s = (e,t,d) \qquad e(\mathbf{mode}(tns)) = \mathbf{read} \qquad tns \in d \qquad e(tns) = tns'}{\langle \mathbf{thaw}(tns), s\rangle \to (e[\mathbf{mode}(tns) \mapsto \mathbf{update}], \langle\!\langle thaw(tns'), t\rangle\!\rangle, d)}\textit{Thaw}$$

Fig. 13. Basic evaluation semantics, roughly defining most of these language constructs to function similarly to their classical definitions. The state, s, of the program is a tuple (e, t, d) of a variable value environment, another state t corresponding to the state in the host language, and finally the set of tensors defined within the current scope, d. We evolve tensor state with $\langle\rangle$ and host state with $\langle\!\langle\rangle\!\rangle$. Several looplets introduce variables into the host state, which may be read when evaluating the **value** node. Lifecycle functions are designed to be implemented and executed in the host language, but these semantics enforce that each function may update state in the host language and flip the mode of the tensor between **read** and **update**.

$$T := \texttt{EXPR} \mid \texttt{STMT}$$

$$E := [\cdot] \mid \mathbf{loop}(T,T,E) \mid \mathbf{block}(E,T\ldots) \mid \mathbf{block}(T,E,T\ldots) \mid \mathbf{sieve}(E,T) \mid \mathbf{assign}(E,T,T) \mid \mathbf{assign}(T,T,E) \mid$$
$$\mathbf{declare}(T,E,T) \mid \mathbf{declare}(T,T,E) \mid \mathbf{call}(E,T\ldots) \mid \mathbf{call}(T,E,T\ldots) \mid \mathbf{access}(T,E,T\ldots) \mid \mathbf{access}(T,T,E\ldots)$$

$$\frac{e(tns) \mapsto tns' \qquad e(\mathbf{mode}(tns)) \mapsto m \qquad \langle\!\langle unfurl(tns', ext, m), t\rangle\!\rangle \Rightarrow tns''}{\langle \mathbf{loop}(i, ext, E[\mathbf{access}(tns, j\ldots, i)]), s\rangle \to \langle \mathbf{loop}(i, ext, E[\mathbf{access}(tns'', j\ldots, i)]), s\rangle}\ \textit{Unfurl}$$

$$\frac{}{\langle \mathbf{loop}(i, ext, E[\mathbf{access}(\mathbf{run}(body), j\ldots, i)]), s\rangle \to \langle \mathbf{loop}(i, ext, E[\mathbf{access}(body, j\ldots)]), s\rangle}\ \textit{Run}$$

$$\frac{e(i) = i' \qquad \langle\!\langle seek(i'), t\rangle\!\rangle \to t'}{\langle E[\mathbf{access}(\mathbf{lookup}(seek, body), j\ldots, i)], (e, t, d)\rangle \to \langle E[\mathbf{access}(body, j\ldots)], (e, t', d)\rangle}\ \textit{Lookup}$$

$$\frac{}{\langle \mathbf{loop}(i, \mathbf{extent}(a,b), E[\mathbf{assign}(\mathbf{access}(\mathbf{run}(body), j\ldots, i), op, rhs)]), s\rangle \to \langle \mathbf{loop}(i, \mathbf{extent}(a,b), E[\mathbf{sieve}(i == a, \mathbf{assign}(\mathbf{access}(body, j\ldots), op, rhs))]), s\rangle}\ \textit{AcceptRun}$$

$$\frac{\langle\!\langle cond, t\rangle\!\rangle \Rightarrow true}{\langle E[\mathbf{access}(switch(cond, head, tail), i\ldots)], s\rangle \to \langle E[\mathbf{access}(head, i\ldots)], s\rangle}\ \textit{SwitchTrue} \qquad \frac{\langle\!\langle cond, t\rangle\!\rangle \Rightarrow false}{\langle E[\mathbf{access}(switch(cond, head, tail), i\ldots)], s\rangle \to \langle E[\mathbf{access}(tail, i\ldots)], s\rangle}\ \textit{SwitchFalse}$$

$$\frac{}{\langle \mathbf{loop}(i, \mathbf{extent}(a,b), E[\mathbf{access}(\mathbf{phase}(\mathbf{extent}(c,d), body), j\ldots, i)]), s\rangle \to \langle \mathbf{loop}(i, \mathbf{extent}(max(a,c), min(b,d)), E[\mathbf{access}(body, j\ldots, i)]), s\rangle}\ \textit{Phase}$$

$$\frac{\langle\!\langle preamble, t\rangle\!\rangle \to t' \qquad \langle E[body], (e, t', d)\rangle \to (e', t'', d) \qquad \langle\!\langle epilogue, t''\rangle\!\rangle \to t'''}{\langle E[thunk(preamble, body, epilogue)], (e, t, d)\rangle \to (e', t''', d)}\ \textit{Thunk}$$

$$\frac{\langle \mathbf{loop}(i, ext, E[\mathbf{access}(head, j\ldots, i)]), s\rangle \to s'}{\langle \mathbf{loop}(i, ext, E[\mathbf{access}(sequence(head, tail), j\ldots, i)]), s\rangle \to \langle \mathbf{loop}(i, ext, E[\mathbf{access}(tail, j\ldots, i)]), s'\rangle}\ \textit{Sequence} \qquad \frac{\langle node, algebra\rangle \to node'}{\langle E[node], s\rangle \to \langle E[node'], s\rangle}\ \textit{Simplify}$$

$$\frac{\langle\!\langle seek(a), t\rangle\!\rangle \to t'}{\langle \mathbf{loop}(i, \mathbf{extent}(a,b), E[\mathbf{access}(stepper(seek, body, next), j\ldots, i)]), (e, t, d)\rangle \to \langle \mathbf{loop}(i, \mathbf{extent}(a,b), E[\mathbf{access}(stepper(body, next), j\ldots, i)]), (e, t', d)\rangle}\ \textit{StepperSeek}$$

$$\frac{\langle \mathbf{loop}(i, ext, E[\mathbf{access}(body, j\ldots, i)]), (e, t, d)\rangle \to (e', t', d) \qquad \langle\!\langle next, t'\rangle\!\rangle \to t''}{\langle \mathbf{loop}(i, ext, E[\mathbf{access}(stepper(body, next), j\ldots, i)]), s\rangle \to \langle \mathbf{loop}(i, ext, E[\mathbf{access}(stepper(body, next), j\ldots, i)]), (e', t'', d)\rangle}\ \textit{StepperNext}$$

$$\frac{}{\langle \mathbf{loop}(i, \mathbf{extent}(a,b), body), s\rangle \to \langle \mathbf{block}(\mathbf{define}(i, a, body), \mathbf{sieve}(a < b, \mathbf{loop}(i, \mathbf{extent}(a+1, b), body))), s\rangle}\ \textit{Loop}$$

Fig. 14. Looplet evaluation semantics. The state s of the program is a tuple (e, t, d) of a variable value environment, host language state t, and the current tensor scope, d. Note that E is an evaluation context that applies anywhere in the syntax tree. The nonlocal evaluations of looplets are what allow looplets to hoist conditions and subranges out of loops. However, this also means we must specify the priority in which we apply looplet rules, which is as follows: *Thunk* > *Phase* > *Switch* > *Simplify* > *Run* > *Spike* > *Sequence* > *StepperSeek* > *StepperNext* > *Lookup* > *AcceptRun* > *Unfurl* > *Loop* > *Access*. Many looplets, most notably the thunk looplet, introduce variables into the host language environment. While looplets may modify variables they introduce themselves (steppers often increment some state variables), we forbid child looplets from modifying state variables that they didn't introduce. This allows us to treat the **value** node as a constant. The *Simplify* rule references *algebra*, which is our variable defining a set of straightforward simplification rules. These rules include simple properties like $x * 0 \to 0$ to more complicated ones such as constant propagation. We omit the full set of rules for brevity and refer to [7, Figure 5] for examples.

6 EXAMPLE LOWERING

Though Finch programs look as if they are written for dense loops, Finch specializes the code during lowering so that only the necessary elements of structure need to be processed. In Figures 15-16, we illustrate the lowering of a program that sums the upper triangle of an `m` by `n` matrix, `A`.

Input Program:

```
A = Tensor(Dense(SparseList(Element(0.0))), m, n)
s = Tensor(Element(0.0))
@finch begin
  s .= 0.0
  for j = _
    for i = _
      if i <= j
        s[] += A[i, j]
```

Step 1: Normalization To begin, the wrapperization pass replaces `i <= j` with `UpTriMask()[i, j]`. The dimensionalization pass assigns `i`, and `j` the dimensions `m` and `n`, respectively. All accesses are concordant, so, after adding lifecycle statements, we have:

```
T = UpTriMask()
A = Tensor(Dense(SparseList(Element(0.0))), m, n)
s = Tensor(Element(0.0))
@finch begin
  @declare(s, 0.0)
  for j = 1:n
    for i = 1:m
      if T[i, j]
        s[] += A[i, j]
  @freeze(s)
```

Step 2: Declaring s We then begin lowering the program. The `@declare(s, 0.0)` statement results in the initialization of the `val` tensor, `s.lvl.val[1] = 0.0`.

```
s.lvl.val[1] = 0.0
@finch begin
  for j = 1:n
    for i = 1:m
      if UpTriMask()[i, j]
        s[] += A[i, j]
  @freeze(s)
```

Step 3: Unfurling j To process the `j` loop, we unfurl both tensors that access `j`:

```
s.lvl.val[1] = 0.0
@finch begin
  for j = 1:n
    for i = 1:m
      if (t[j])[i]
        s[] += (a[j])[i]
  @freeze(s)
```

```
t = unfurl(UpTriMask()) =
  Lookup(
    body(j) = UpTriMaskCol(j))
a = unfurl(A::DenseLevel) =
  Lookup(
    body(j) = SubFiber(A.lvl.lvl, j))
```

Step 4: Lowering Lookups The Lookup pass inserts a for-loop:

```
s.lvl.val[1] = 0.0
for j = 1:n
  @finch begin
    for i = 1:m
      if UpTriMaskCol(j)[i]
        s[] += SubFiber(A.lvl.lvl, j)[i]
@finch @freeze(s)
```

Step 5: Unfurling i Next, we process the `i` loop. Again, we unfurl both tensors:

```
s.lvl.val[1] = 0.0
for j = 1:n
  @finch begin
    for i = 1:m
      if t[i]
        s[] += a[i]
@finch @freeze(s)
```

```
t = unfurl(UpTriMaskCol(j)) =
  Sequence(
    Phase(stop = j, Run(true)),
    Phase(Run(false)))
a = unfurl(SubFiber(
    A.lvl.lvl::SparseListLevel, j)) =
  Thunk(
    preamble = (q = A.lvl.lvl.ptr[j]),
    Stepper(
      seek = (i) -> (
        q = binarysearch(A.lvl.lvl.idx, i)),
      stop = A.lvl.lvl.idx[q],
      body = Spike(
        body = 0,
        tail = A.lvl.lvl.val[q]),
      next = (q += 1)))
```

Step 6: Lowering Thunk The Thunk pass moves the preamble out of the Thunks and unwraps it:

```
s.lvl.val[1] = 0.0
for j = 1:n
  q = A.lvl.lvl.ptr[j]
  @finch for i = 1:j
    if t[i]
      s[] += a[i]
@finch @freeze(s)
```

```
t = Sequence(
  Phase(stop = j, Run(true)),
  Phase(Run(false)))
a = Stepper(
  seek = (i) -> (
    q = binarysearch(A.lvl.lvl.idx, i)),
  stop = A.lvl.lvl.idx[q],
  body = Spike(
    body = 0,
    tail = A.lvl.lvl.val[q]),
  next = (q += 1))
```

Step 7: Lowering Sequence The Sequence pass introduces separate loops for each phase:

```
s.lvl.val[1] = 0.0
for j = 1:n
  q = A.lvl.lvl.ptr[j]
  @finch for i = 1:j
    if t_1[i]
      s[] += a[i]
  @finch for i = j+1:m
    if t_2[i]
      s[] += a[i]
@finch @freeze(s)
```

```
t_1 = Run(true)
t_2 = Run(false)
a = Stepper(
  seek = (i) -> (
    q = binarysearch(A.lvl.lvl.idx, i)),
  stop = A.lvl.lvl.idx[q],
  body = Spike(
    body = 0,
    tail = A.lvl.lvl.val[q]),
  next = (q += 1))
```

Step 8: Lowering Run The Run pass simply replaces runs with their value:

```
s.lvl.val[1] = 0.0
for j = 1:n
  q = A.lvl.lvl.ptr[j]
  @finch for i = 1:j
    if true
      s[] += a[i]
  @finch for i = j+1:m
    if false
      s[] += a[i]
@finch @freeze(s)
```

```
a = Stepper(
  seek = (i) -> (
    q = binarysearch(A.lvl.lvl.idx, i)),
  stop = A.lvl.lvl.idx[q],
  body = Spike(
    body = 0,
    tail = A.lvl.lvl.val[q]),
  next = (q += 1))
```

Step 9: Simplify The simplification pass removes the if statement in the first loop and removes the second loop:

```
s.lvl.val[1] = 0.0
for j = 1:n
  q = A.lvl.lvl.ptr[j]
  @finch for i = 1:j
    s[] += a[i]
@finch @freeze(s)
```

```
a = Stepper(
  seek = (i) -> (
    q = binarysearch(A.lvl.lvl.idx, i)),
  stop = A.lvl.lvl.idx[q],
  body = Spike(
    body = 0,
    tail = A.lvl.lvl.val[q]),
  next = (q += 1))
```

Step 10: Lowering Steppers The Stepper pass introduces a while loop:

```
s.lvl.val[1] = 0.0
for j = 1:n
  q = A.lvl.lvl.ptr[j]
  k = 1
  while k < j
    k_2 = A.lvl.lvl.idx[q]
    @finch for i = k:min(k_2, j)
      s[] += a[i]
    q += 1
@finch @freeze(s)
```

```
a = Spike(
    body = 0,
    tail = A.lvl.lvl.val[q])
```

Step 11: Lowering Spike The Spike pass acts like Sequence combined with a Run:

```
s.lvl.val[1] = 0.0
for j = 1:n
  q = A.lvl.lvl.ptr[j]
  k = 1
  while k < j
    k_2 = A.lvl.lvl.idx[q]
    @finch for i = k:min(k_2, j)
      s[] += 0
    i = k_2
    @finch if i < j
      s[] += A.lvl.lvl.val[q]
    q += 1
@finch @freeze(s)
```

Fig. 15. Example lowering of a Finch program, continued in Figure 16.

Step 12: Simplify The simplify pass recognizes that addition of **0** is a no-op:

```
s.lvl.val[1] = 0.0
for j = 1:n
  q = A.lvl.lvl.ptr[j]
  k = 1
  while k < j
    k_2 = A.lvl.lvl.idx[q]
      i = k_2
      if i < j
        s[] += A.lvl.lvl.val[q]
    q += 1
  @finch @freeze(s)
```

Step 13: Lower Freeze Finally, the final `freeze` is a no-op and we obtain:

```
s.lvl.val[1] = 0.0
for j = 1:n
  q = A.lvl.lvl.ptr[j]
  k = 1
  while k < j
    k_2 = A.lvl.lvl.idx[q]
      i = k_2
      if i < j
        s[] += A.lvl.lvl.val[q]
    q += 1
```

Fig. 16. Example lowering of a Finch program, continued from 15. The final program accesses only the upper triangle of A, though the original code looks as though it loops over all `i` and `j`.

7 CASE STUDIES

We evaluate Finch on a broad set of applications to showcase it's efficiency, flexibility, and expressiveness. All of our implementations highlight the benefits of data structure and algorithm co-design. Our implementation of sparse-sparse-matrix multiply (SpGEMM) translates classical lessons from sparse performance engineering into the language of Finch, using temporaries and randomly-accessible workspace formats to efficiently implement the three main approaches. Our study of sparse-matrix-dense-vector multiply (SpMV) highlights the benefits of precise structural specialization. Our studies of image morphology and graph applications show how Finch's programming model can express more complex real-world kernels.

All experiments were run on a single core of a 12-core 2-socket Intel Xeon CPU E5-2680 v3 running at 2.50GHz with 128GB of memory. Finch is implemented in Julia v1.9, targeting LLVM through Julia. All timings are the minimum of 10,000 runs or 5s of measurement, whichever happens first.

7.1 Sparse Matrix-Vector Multiply (SpMV)

Sparse matrix-vector multiply (SpMV) has a wide range of applications and has been thoroughly studied [68, 107]. Because SpMV is bandwidth bound, many formats have been proposed to reduce the footprint [64]. The wide range of applications results in a wide range of tensor structures, making it an effective kernel to demonstrate the utility of our programming model.

```
y .= 0
for j = _, i = _
  y[i] += A[i, j] * x[j]
```

```
y .= 0
for j = _, i = _
  y[j] += A[i, j] * x[i]
```

```
y .= 0
for j = _
  let x_j = x[j]
    y_j .= 0
    for i = _
      let A_ij = A[i, j]
        y[i] += x_j * A_ij
        y_j[] += A_ij * x[i]
    #D is the diagonal
    y[j] += y_j[] + D[j] * x_j
```

Fig. 17. Finch row-major, column-major and symmetric SpMV Programs. Note that the upper triangle of the input is pre-computed for the symmetric program. Reads to the canonical triangle are reused with a **define** statement, and the results are written to both relevant locations using multiple outputs.

Figure 18 displays the performance of SpMV measured relative to TACO. We varied both the data formats and the SpMV algorithm. We display the best Finch format and algorithm among all the formats and algorithms listed in Figure 17, wherever each format and algorithm are applicable. Precisely which Finch format performed best on which matrices is shown in the figure. We compare against TACO (best of row or column-major), Julia's standard library (column major), baseline Finch (best of row or column-major), Eigen (row-major) [48], MKL (row-major) [1], and CORA (unscheduled, row-major) [40]. Our test suite is the union of datasets from three previous papers: the matrices used by Ahrens et al. to test a variable block row format partitioning strategy [6], Kjolstad et al. to test the TACO library [60], and Leskovec et al. to evaluate graph clustering algorithms [66]. We left out two very large matrices (`Janna/Emilia_923` and `Janna/Geo_1438`); the remaining matrices in our dataset had a maximum of 12 million nonzeros. We also added some

synthetic matrices, $10,000 \times 10,000$ banded matrices with bandwidth 5, 30, and 100, a 1024×1024 upper triangular matrix, and a $1,000,000 \times 1,000,000$ reverse permutation matrix.

Finch commonly introduces tradeoffs between branching and more complicated loop structures and the benefits of such specialization. Specialization is better in cases where the specialized routine is much faster and the common case (such as the zero region of a sparse tensor, which becomes a no-op). However, specialization introduces many different branches, and complicates the bounds of loops. For example, in our symmetric kernel, we chose to pre-compute the upper triangle of the input matrix, rather than calculate it on the fly using a mask expression such as `i < j`, which would change the exit condition of the inner loop. The option to de-specialize certain conditional expressions is another example of how Finch can widen the design space for structured operators.

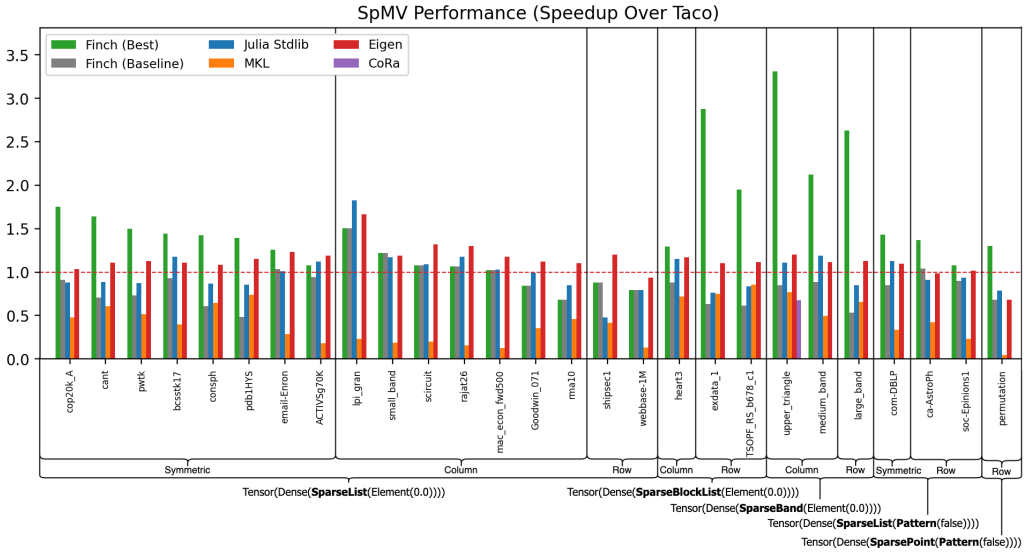


Fig. 18. Performance of SpMV algorithms, organized by the best performing Finch format. The displayed Finch performance is the fastest among the formats we tested. Programs are from Figure 17. "finch_baseline" is the faster of row major or column major "SparseList".

Our result shows that different formats perform better on different matrices, and that Finch can be used to exploit these formats effectively. We found that the SpMV performance was superior for the level format that best paralleled the structure of the tensor. The best Finch format had a geomean speedup of 1.27 over TACO. Matrices with a clear blocked structure like exdata_1, TSOPF_RS_b678_c1, and heart3 performed notably well with the SparseBlockList format with speedups of 2.75, 1.80, and 1.20 relative to TACO, while the baseline format was slightly slower than TACO. Furthermore, the synthetic banded matrices we constructed performed the best with the SparseBand matrix, in particular with the large_band and the medium_band matrices having a speedup of 2.50 and 2.02 relative to TACO, while the baseline format had minor slowdowns relative to TACO. On our triangular matrix, Finch had a speedup of 3.04 over TACO, outperforming even CORA, which was designed for ragged tensors but whose optimizations were targeted more towards cache blocking than to the specific structure of the tensor. The Pattern leaf level performed better than the Element leaf level for representing Boolean graph matrices. For example, the SparseList-Pattern format for `ca-AstroPh` resulted in a speedup of 1.17, while the baseline Finch format resulted only achieved 1.04. Our results clearly demonstrate the utility of being able to vary both the algorithm and the format to match the structure of the tensor.

7.2 Sparse-Sparse Matrix Multiply (SpGEMM)

We compute the $M \times N$ sparse matrix C as the product of $M \times K$ and $K \times N$ sparse matrices A and B. There are three main approaches to SpGEMM [105, Section 2.2]. The inner-products algorithm takes dot products of corresponding rows and columns, while the outer-products algorithm sums the outer products of corresponding columns and rows. Gustavson's algorithm sums the rows of B scaled by the corresponding nonzero columns in each row of A. Inner-products is known to be asymptotically less efficient than the others, as we must do a merge operation to compute each of the $O(MN)$ entries in the output [8]. We will show that our ability to implement these latter methods exceeds that of TACO, translating to asymptotic benefits.

```
@finch begin
  C .= 0
  for j=_
    for i=_
      for k=_
        C[i, j] += AT[k, i] * B[k, j]
  return C
```

```
w = Tensor(SparseByteMap(Element(0)))
@finch begin
  C .= 0
  for j=_
    w .= 0
    for k=_
      for i=_
        w[i] += A[i, k] * B[k, j]
    for i=_
      C[i, j] = w[i]
```

```
w = Tensor(SparseHash(SparseHash(Element(0))))
@finch begin
  w .= 0
  for k=_
    for j=_
      for i=_
        w[i, j] += A[i, k] * BT[j, k]
  C .= 0
  for j=_, i=_
    C[i, j] = w[i, j]
```

Fig. 19. Inner Products, Gustavson's, and Outer Products matrix multiply in Finch

Figure 19 implements all three approaches in Finch, and Figure 20 compares the performance of Finch to TACO, Eigen, and MKL on the matrices of Zhang et al. [105]. Note that these algorithms mainly differ in their loop order, but that different data structures can be used to support the various access patterns induced. In our Finch implementation of outer products, we use a sparse hash table, as it is fully-sparse and randomly accessible. Since, TACO does not support multidimensional sparse workspaces, its outer products uses a dense intermediate, which leads to an asymptotic slow down shown in Figure 20. Similarly, although a sparse bytemap has a dense memory footprint, we use it in our Finch implementation of Gustavson's for the smaller $O(N)$ intermediate. We note that the bytemap format in TACO's Gustavson's implementation is hard-wired, whereas Finch's programming model allows us to write algorithms with explicit temporaries and transpositions. Without such hard wiring, TACO would have to use a dense intermediate to support random

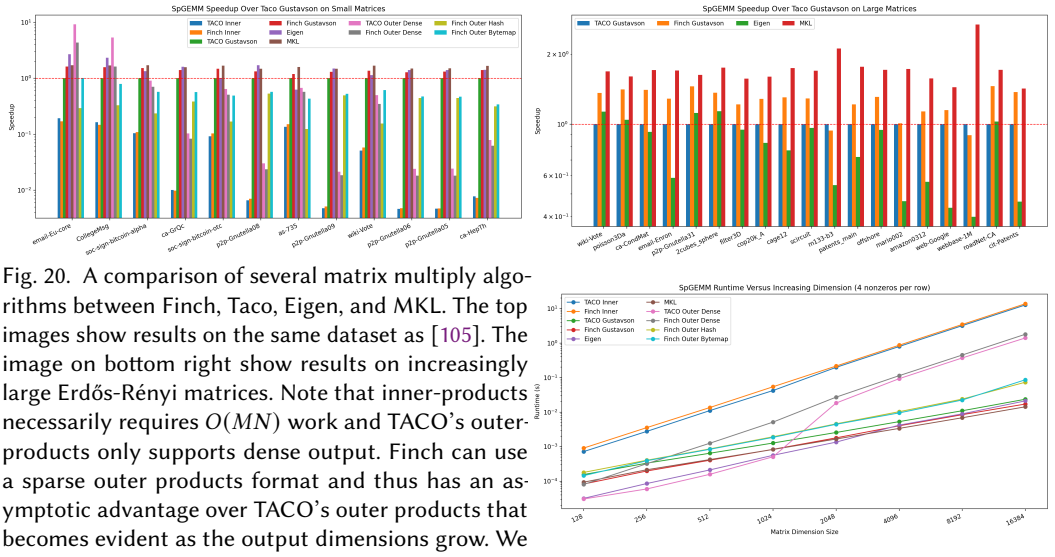


Fig. 20. A comparison of several matrix multiply algorithms between Finch, Taco, Eigen, and MKL. The top images show results on the same dataset as [105]. The image on bottom right show results on increasingly large Erdős-Rényi matrices. Note that inner-products necessarily requires $O(MN)$ work and TACO's outer-products only supports dense output. Finch can use a sparse outer products format and thus has an asymptotic advantage over TACO's outer products that becomes evident as the output dimensions grow. We use only Gustavson's algorithm on larger matrices.

writes, which TACO would then propagate to the output, turning it dense and leading to the same asymptotic results as in the case of outer products. As depicted in Figure 20, Finch achieves comparable performance with TACO on smaller matrices when we use the same datastructures, and significant improvements when we use better datastructures. Finch outperforms TACO overall, with a geomean speedup of 1.30. Finch and TACO both outperform Eigen by a significant margin, as Eigen is designed for usability but is not heavily optimized.

7.3 Graph Analytics

We used Finch to implement both Breadth-first search (BFS) and Bellman-Ford single-source shortest path. Our BFS implementation and graphs datasets are taken from Yang et al. [102], including both road networks and scale-free graphs (bounded node degree vs. power law node degree).

Direction-optimization [15] is crucial for achieving high BFS performance in such scenarios, switching between push and pull traversals to efficiently explore graphs. Push traversal visits the neighbors of each frontier node, while pull traversal visits every node and checks to see if it has a neighbor in the frontier. The advantage of pull traversal is that we may terminate our search once we find a node in the frontier, saving time in the event the push traversal were to visit most of the graph anyway. Early break is the critical part of control flow in this algorithm, though the algorithms also require different loop orders, multiple outputs, and custom operators. Finch performs well because it can directly express algorithms comparable to competitive libraries.

```
V = Tensor(Dense(Element(false)))
P = Tensor(Dense(Element(0)))
F = Tensor(SparseByteMap(Pattern()))
_F = Tensor(SparseByteMap(Pattern()))
A = Tensor(Dense(SparseList(Pattern())))
AT = Tensor(Dense(SparseList(Pattern())))

function bfs_push(_F, F, A, V, P)
  @finch begin
    _F .= false
    for j=_, k=_
      if F[j] && A[k, j] && !(V[k])
        _F[k] |= true
        P[k] <<choose(0)>>= j
    return _F
```

```
function bfs_pull(_F, F, AT, V, P)
  p = ShortCircuitScalar{0}()
  @finch begin
    _F .= false
    for k=_
      if !V[k]
        p .= 0
        for j=_
          if F[follow(j)] && AT[j, k]
            p[] <<choose(0)>>= j
        if p[] != 0
          _F[k] |= true
          P[k] = p[]
    return _F
```

```
_D = Tensor(Dense(Element(Inf)), n)
D = Tensor(Dense(Element(Inf)), n)
function bellmanford(A, _D, D, _F, F)
  @finch begin
    F .= false
    for j = _
      if _F[j]
        for i = _
          let d = _D[j] + A[i, j]
            D[i] <<min>>= d
            F[i] |= d < _D[i]
```

Fig. 21. Graph Applications written in Finch. Note that parents are calculated separately for Bellman-Ford. The *choose*(z) operator is a GraphBLAS concept which returns any argument that is not z.

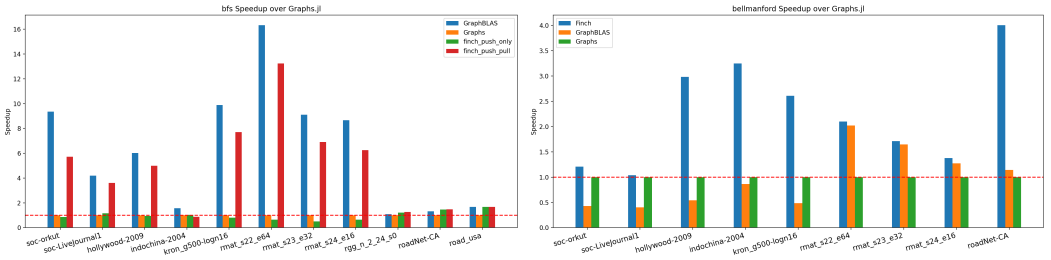


Fig. 22. Performance of graph apps across various tools. finch_push_only exclusively utilizes push traversal, while finch_push_pull applies direction-optimization akin to GraphBLAS. Finch's support for push/pull traversal and early break facilitates direction-optimization. Among GraphBLAS's five variants for Bellman-Ford, we selected LAGraph_BF_full1a, consistently the fastest with our graphs. We did not include Bellman-Ford results for graphs with high diameter as they timed out (> 1 hour).

Figure 22 compares performance to Graphs.jl, a Julia library, and the LAGraph Library, which implements graph algorithms with sparse linear algebra using GraphBLAS [70]. On BFS, Finch is competitive even with the hardwired optimizations of GraphBLAS, a geomean slowdown of 1.22. Direction-optimization notably enhances performance for scale-free graphs. On Bellman-Ford (with path lengths and shortest-path tree), Finch's support for multiple outputs, sparse inputs, and masks leads to a geomean speedup of 2.47 over GraphBLAS. Appendix B displays code for BFS and Bellman-Ford in Finch (57 and 50 LOC) and LAGraph (215 and 227 LOC), and we invite readers to compare the clarity of the algorithms.

7.4 Image Morphology

Some image processing pipelines stand to benefit from structured data processing [36]. We focus on binary image morphology and the logical transformation of binary images and masks. We consider two operations: binary erosion (computing a mask), and a masked histogram (using a mask to avoid work). We use images are all binary, either by design or having been thresholded.

Finch allows us choose our datastructure, so we may choose to use either a dense representation with bytes ($Dense(Element(0x00))$), a bit-packed representation ($Dense(Element(UInt64))$), or a run-length encoded representation that represents runs of true or false ($SparseRunList(Pattern())$). All of these have their advantages. The dense representation induces the least overhead, the bit-packed representation can take advantage of bitwise binary ops, and the run-length encoded version only uses memory and compute when the pattern changes.

Similarly, since Finch let's us choose our algorithm we can implement erosion in a few ways. The erosion operation turns off a pixel unless all of it's neighbors are on. This can be used to shrink the boundaries of a mask, and remove point instances of noise [41]. This introduces three instances of structure in the control flow: the mask, the padding of inputs, and the convolutional filter. We focused on the filter. We might understand the filter as a structured tensor of circular shifts, or we might understand each shifted view of the data in an unrolled stencil computation as a structured tensor, or a two part stencil where we compute the horizontal then vertical part of the stencil. We experimented with these options and found that the last approach performed best, due to fitting the storage formats while reducing the amount of work with intermediate temporaries. Figure 23 displays example erosion algorithms for bitwise or run-length-encoded algorithms.

We compared against OpenCV on four datasets. We randomly selected 100 images from the MNIST [65] and Omniglot [63] character recognition datasets, as well as a dataset of human line drawings [38]. We also hand-selected a subset of mask images (these images were less homogeneous, so we listed them in Appendix C) from a digital image processing textbook [46]. All images were thresholded, and we also include versions of the images that have been magnified before thresholding, to induce larger constant regions. In our erosion task, the SparseRunList format performs the best as it is asymptotically faster and uses less memory, leading to a 19.5X speedup over OpenCV on the sketches dataset, which becomes arbitrarily large as we magnify the images (here shown as 266X). Finch achieves these speedups by exploiting structured sparsity to straightforwardly do less work than OpenCV's more naive dense implementation, which must still read most of an image or mask even when it is unnecessary. However, we believe the 51.6x on MNIST is due to calling overhead in OpenCV. The bitwise kernels were effective as well, and would be more effective on datasets with less structure. A strength of Finch is that it supports structured datasets, even over bitwise operations, allowing us to implement the bitwise kernel and then mask it.

We also implemented a histogram kernel. We used an indirect access into the output to implement this (Figure 23), something not many sparse frameworks support. We compare to OpenCV since the OpenCV histogram function also accepts a mask. If we use $SparseRunList(Pattern())$ for our mask, we can reduce the branching in the masked kernel and get better performance. The improvements

Wordwise Erosion:

```
output .= false
for y = _
  tmp .= false
  for x = _
    tmp[x] = coalesce(input[x, ~(y-1)], true) & input[x, y] &
    ↪  coalesce(input[x, ~(y+1)], true)
  for x = _
    output[x, y] = coalesce(tmp[~(x-1)], true) & tmp[x] &
    ↪  coalesce(tmp[~(x+1)], true)
```

Masked Histogram:

```
bins .= 0
for x=_
  for y=_
    if mask[y, x]
      bins[div(img[y, x], 16) + 1] += 1
```

Bitwise Erosion:

```
output .= 0
for y = _
  tmp .= 0
  for x = _
    if mask[x, y]
      tmp[x] = coalesce(input[x, ~(y-1)], 0xFFFFFFFF) & input[x,
      ↪  y] & coalesce(input[x, ~(y+1)], 0xFFFFFFFF)
  for x = _
    if mask[x, y]
      let tl = coalesce(tmp[~(x-1)], 0xFFFFFFFF), t = tmp[x], tr =
      ↪  coalesce(tmp[~(x+1)], 0xFFFFFFFF)
        let res = ((tr << (8 * sizeof(UInt) - 1)) | (t >> 1)) & t &
        ↪  ((t << 1) | (tl >> (8 * sizeof(UInt) - 1)))
          output[x, y] = res
```

Fig. 23. Two approaches to erosion in Finch. The *coalesce* function defines the out of bounds value. On left, the naive approach. On $SparseRunList(Pattern())$ inputs, this only performs operations at the boundaries of constant regions. On right, a bitwise approach, using a mask to limit work to nonzero blocks of bits.

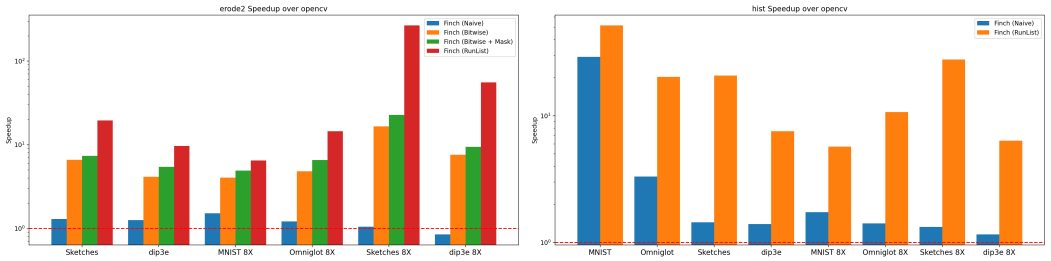


Fig. 24. Performance of Finch on image morphology tasks. On left, we run 2 iterations of erosion. On right, we run a masked histogram. We display the geomean speedup within each dataset.

with SparseRunList are seen in the histogram task too, as it allows us to mask off contiguous regions of computation, instead of individual pixels, reducing the branches and leading to a significant speedup (20.3x on Omniglot and 20.8x on sketches). In a low compute task such as a histogram, skipping many reads for the mask via structured sparsity can lead to huge speedups.

8 RELATED WORK

The related work on tensor languages and libraries spans several areas, from libraries to languages, from dense to structured computation.

Libraries for Dense Data: Many libraries specialize in dense computations. Perhaps the most well-known example is NumPy [50], and a classic example is the BLAS, though several BLAS routines are specialized to symmetric, hermitian, and triangular matrices [9]. Many research projects have advanced on BLAS, such as BatchedBlas and BLIS [37, 94].

Libraries for Structured Data: Many libraries support BLAS plus a few sparse tensor types, typically CSR, CSC, BCSR, Banded, and COO. Examples include SciPy [97], PETSc [5], Armadillo [79], OSKI [99], Cyclops [87], MKL [1], and Eigen [48]. There are even libraries for very specific kernels and format combinations, such as SPLATT [85] (MTTKRP on CSF). Several of these libraries also feature some graph or mesh algorithms built on sparse matrices. The GraphBLAS [58] supports primitive semiring operations (operations beyond $(+, *)$, such as $(min, +)$ multiplication) which can be composed to enable graph algorithms, some of which are collected in LAGraph [70]. Similarly, the

MapReduce and Hadoop platforms support operations on indexed collections [34], and have been used to support graph algorithms in the GBASE library[57]. Several machine learning frameworks support some sparse tensors and operations, most notably TorchSparse[91, 92].

Compilers for Dense Data: Outside of general purpose compilers, many compilers have been developed for optimizing dense data on a variety of control flow. Perhaps the most well known example is Halide [78] and its various descendant such as TVM [28], Exo [55], Elevate [49], and ATL [67]. These languages typically support most control flow except for an early break though some don't support arbitrary reading/writing or even indirect accesses. Several polyhedral languages, such as Polly [47], Tiramisu [11], CHiLL [27], Pluto [22], and AlphaZ [104] offer similar capabilities in terms of control flow though they often support more irregular regions that the polyhedral framework supports. These are based on ISL [96]. The density of this research represents the density of support for dense computation.

Compilers for Structured Data: Several compilers exist for several types of structured data, often featuring separate languages for the storage of the structured data and the computation. The TACO compiler originally supported just plain Einsum computations [60], but has been extended several times to support (single dimensional) local tensors [59], imperfectly nested loops [35], breaks via semi-rings [51], windowing and tiling [82], and convolution [100], and compilation in MLIR [20], all as separate extensions. Similarly, TACO originally support just dense and CSF like N dimensional structures, but was extended independently to support COO like structures [30], and tree like structures [29], as separate extensions. SparseTIR is a similar system supporting combined sparse formats (including block structures) [103]. The SDQL language offers a similar level of control flow [83], but only on sparse hash tables. Similarly, SDQL has been extended with a system that allows one to specify formats as queries on a set of base storage types [81] and separately by another system that describes static symmetries and other structures as predicates [44]. The Taichi language focuses on a single sparse data structure made from dense blocks, bit-masks, and pointers [53]. The sparse polyhedral framework builds on CHiLL for the purpose of generating inspector/executor optimizations [89] though the branch of this work that specifies sparse formats separately from the computation (otherwise they are inlined into the computation manually) seems to apply mainly to Einsums [106]. Second to last, SQL's classical physical/logical distinction is the classic program/format distinction, and SQL supports a huge variety of control flow constructs [32, 62]. However, many SQL or dataframe systems rely on b-trees, columnar, or hash tables, with only a few systems, such as Vectorwise [21], LaraDB [54], GMAP [93], or SciDB [88] building physical layouts with other constructs based in tensor programming. However, tensor based databases are a new focus given the rise of mixed ML/DB pipelines [14, 69]. Lastly, SPIRAL focuses on recursively defined datastructures and recursively define linear algebra, and can therefore express a structure and computation that none of the systems mentioned above can: a Cooley–Tukey FFT [42, 43].

Other Architectures: Sparse compilers have been extended to many architectures. An extension of TACO supports GPU [82], Cyclops [86, 87] and SPDistal [101] support distributed memory, and the Sparse Abstract Machine [52] supports custom hardware. We believe that supporting control flow is the first step towards architectural support beyond unstructured sparsity.

9 CONCLUSION

Finch automatically specializes flexible control flow to diverse data structures, facilitating productive algorithmic exploration, flexible tensor programming, and efficient high-level interfaces for a wider variety of applications than ever before.

10 DATA AVAILABILITY STATEMENT

The Finch compiler and the benchmarks used to construct this paper are both currently available as open-source software, and we plan to make them available as an artifact to the OOPSLA Artifact Evaluation Committee. We will provide instructions to replicate all results of the paper.

REFERENCES

[1] 2024. Developer Reference for Intel® oneAPI Math Kernel Library for Fortran. (April 2024). https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-fortran/2024-0/overview.html

[2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. 265–283. https://www.usenix.org/system/files/conference/osdi16/osdi16-abadi.pdf

[3] Hameer Abbasi. 2023. Plans for new sparse compilation backend · pydata/sparse · Discussion #618. https://github.com/pydata/sparse/discussions/618

[4] Harold Abelson and Gerald Jay Sussman. 1996. *Structure and Interpretation of Computer Programs*. The MIT Press. https://library.oapen.org/handle/20.500.12657/26092 Accepted: 2019-01-17 23:55.

[5] SHRIRANG ABHYANKAR, GETNET BETRIE, DANIEL A MALDONADO, LOIS C MCINNES, BARRY SMITH, and HONG ZHANG. [n. d.]. PETSc DMNetwork: A Scalable Network PDE-Based Multiphysics Simulator. ([n. d.]).

[6] Willow Ahrens and Erik G. Boman. 2021. On Optimal Partitioning For Sparse Matrices In Variable Block Row Format. https://doi.org/10.48550/arXiv.2005.12414 arXiv:2005.12414 [cs].

[7] Willow Ahrens, Daniel Donenfeld, Fredrik Kjolstad, and Saman Amarasinghe. 2023. Looplets: A Language for Structured Coiteration. In *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization (CGO)*. Association for Computing Machinery, New York, NY, USA, 41–54. https://doi.org/10.1145/3579990.3580020

[8] Willow Ahrens, Fredrik Kjolstad, and Saman Amarasinghe. 2022. Autoscheduling for sparse tensor algebra with an asymptotic cost model. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery, New York, NY, USA, 269–285. https://doi.org/10.1145/3519939.3523442

[9] E. Anderson, Z. Bai, C. Bischof, L. S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. 1999. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898719604

[10] J. W. Backus, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, P. B. Sheridan, H. Stern, I. Ziller, R. A. Hughes, and R. Nutt. 1957. The FORTRAN automatic coding system. In *Papers presented at the February 26-28, 1957, western joint computer conference: Techniques for reliability (IRE-AIEE-ACM '57 (Western))*. Association for Computing Machinery, New York, NY, USA, 188–198. https://doi.org/10.1145/1455567.1455599

[11] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. 2019. Tiramisu: a polyhedral compiler for expressing fast and portable code. In *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO 2019)*. IEEE Press, Washington, DC, USA, 193–205.

[12] S Balay, S Abhyankar, Mark F Adams, J Brown, P Brune, K Buschelman, L Dalcin, A Dener, V Eijkhout, W Gropp, and others. 2020. *PETSc Users Manual (Rev. 3.13)*. Technical Report. Argonne National Lab.(ANL), Argonne, IL (United States).

[13] Jérémy Barbay, Alejandro López-Ortiz, Tyler Lu, and Alejandro Salinger. 2010. An experimental investigation of set intersection algorithms for text searching. *ACM Journal of Experimental Algorithmics* 14 (Jan. 2010), 7:3.7–7:3.24. https://doi.org/10.1145/1498698.1564507

[14] Peter Baumann, Dimitar Misev, Vlad Merticariu, and Bang Pham Huu. 2021. Array databases: Concepts, standards, implementations. *Journal of Big Data* 8 (2021), 1–61.

[15] Scott Beamer, Krste Asanović, and David Patterson. 2012. Direction-optimizing breadth-first search. In *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–10.

[16] Robert M Bell and Yehuda Koren. 2007. Lessons from the Netflix prize challenge. *Acm Sigkdd Explorations Newsletter* 9, 2 (2007), 75–79. Publisher: ACM New York, NY, USA.

[17] Gilbert Louis Bernstein, Chinmayee Shah, Crystal Lemire, Zachary Devito, Matthew Fisher, Philip Levis, and Pat Hanrahan. 2016. Ebb: A DSL for physical simulation on CPUs and GPUs. *ACM Transactions on Graphics (TOG)* 35, 2 (2016), 1–12.

[18] J. Bezanson, A. Edelman, S. Karpinski, and V. Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (Jan. 2017), 65–98. https://doi.org/10.1137/141000671

[19] Jeff Bezanson, Stefan Karpinski, Viral B. Shah, and Alan Edelman. 2012. Julia: A Fast Dynamic Language for Technical Computing. *arXiv:1209.5145 [cs]* (Sept. 2012). http://arxiv.org/pdf/1209.5145.pdf arXiv: 1209.5145.

[20] Aart Bik, Penporn Koanantakool, Tatiana Shpeisman, Nicolas Vasilache, Bixia Zheng, and Fredrik Kjolstad. 2022. Compiler Support for Sparse Tensor Computations in MLIR. *ACM Transactions on Architecture and Code Optimization* 19, 4 (Sept. 2022), 50:1–50:25. https://doi.org/10.1145/3544559

[21] PA Boncz and M Zukowski. 2012. Vectorwise: Beyond column stores. *IEEE Data Engineering Bulletin* 35, 1 (2012), 21–27.

[22] Uday Bondhugula, Albert Hartono, J Ramanujam, and P Sadayappan. 2008. Pluto: A practical and fully automatic polyhedral program optimization system. In *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation (PLDI 08), Tucson, AZ (June 2008)*. Citeseer.

[23] Gary Bradski, Adrian Kaehler, and others. 2000. OpenCV. *Dr. Dobb's journal of software tools* 3, 2 (2000).

[24] Aydin Buluç, Tim Mattson, Scott McMillan, José Moreira, and Carl Yang. 2017. Design of the GraphBLAS API for C. In *2017 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*. IEEE, 643–652.

[25] Keaton J. Burns, Geoffrey M. Vasil, Jeffrey S. Oishi, Daniel Lecoanet, and Benjamin P. Brown. 2020. Dedalus: A flexible framework for numerical simulations with spectral methods. *Physical Review Research* 2, 2 (April 2020), 023068. https://doi.org/10.1103/PhysRevResearch.2.023068 _eprint: 1905.10388.

[26] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A Recursive Model for Graph Mining. In *Proceedings of the 2004 SIAM International Conference on Data Mining (SDM)*. Society for Industrial and Applied Mathematics, 442–446. https://doi.org/10.1137/1.9781611972740.43

[27] Chun Chen, Jacqueline Chame, and Mary Hall. 2008. A framework for composing high-level loop transformations. *Technical Report 08–897, USC Computer Science Technical Report* (2008).

[28] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594. https://www.usenix.org/conference/osdi18/presentation/chen

[29] Stephen Chou and Saman Amarasinghe. 2022. Compilation of dynamic sparse tensor algebra. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (Oct. 2022), 175:1408–175:1437. https://doi.org/10.1145/3563338

[30] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format abstraction for sparse tensor algebra compilers. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (Oct. 2018), 123:1–123:30. https://doi.org/10.1145/3276493

[31] Tri Dao, Beidi Chen, Nimit S Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. 2022. Monarch: Expressive structured matrices for efficient and accurate training. In *International Conference on Machine Learning*. PMLR, 4690–4721.

[32] Chris J Date. 1989. *A Guide to the SQL Standard*. Addison-Wesley Longman Publishing Co., Inc.

[33] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. *ACM Trans. Math. Software* 38, 1 (Nov. 2011), 1–25. https://doi.org/10.1145/2049662.2049663

[34] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: simplified data processing on large clusters. *Commun. ACM* 51, 1 (Jan. 2008), 107–113. https://doi.org/10.1145/1327452.1327492

[35] Adhitha Dias, Kirshanthan Sundararajah, Charitha Saumya, and Milind Kulkarni. 2022. SparseLNR: accelerating sparse tensor computations using loop nest restructuring. In *Proceedings of the 36th ACM International Conference on Supercomputing*. ACM, Virtual Event, 1–14. https://doi.org/10.1145/3524059.3532386

[36] Daniel Donenfeld, Stephen Chou, and Saman Amarasinghe. 2022. Unified Compilation for Lossless Compression and Sparse Computing. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 205–216. https://doi.org/10.1109/CGO53902.2022.9741282

[37] J. Dongarra, S. Hammarling, N. Higham, S. D. Relton, P. Valero-Lara, and M. Zounon. 2017. The Design and Performance of Batched BLAS on Modern High-Performance Computing Systems. *Procedia Computer Science* 108 (Jan. 2017), 495–504. https://doi.org/10.1016/j.procs.2017.05.138

[38] Mathias Eitz, James Hays, and Marc Alexa. 2012. How do humans sketch objects? *ACM Transactions on Graphics* 31, 4 (July 2012), 44:1–44:10. https://doi.org/10.1145/2185520.2185540

[39] Pratik Fegade. 2022. The CoRa Tensor Compiler: Compilation for Ragged Tensors with Minimal Padding. https://doi.org/10.5281/zenodo.6326456

[40] Pratik Fegade, Tianqi Chen, Phillip Gibbons, and Todd Mowry. 2022. The CoRa Tensor Compiler: Compilation for Ragged Tensors with Minimal Padding. In *Proceedings of Machine Learning and Systems*, D. Marculescu, Y. Chi, and C. Wu (Eds.), Vol. 4. 721–747. https://proceedings.mlsys.org/paper_files/paper/2022/file/afe8a4577080504b8bec07bbe4b2b9cc-Paper.pdf

[41] Robert Fisher, Simon Perkins, Ashley Walker, and Erik Wolfart. 1996. Hypermedia image processing reference. *England: John Wiley & Sons Ltd* (1996), 118–130.

[42] Franz Franchetti, Frédéric de Mesmay, Daniel McFarlin, and Markus Püschel. 2009. Operator language: A program generation framework for fast kernels. In *IFIP Working Conference on Domain-Specific Languages*. Springer, 385–409.

[43] Franz Franchetti, Tze Meng Low, Doru Thom Popovici, Richard M. Veras, Daniele G. Spampinato, Jeremy R. Johnson, Markus Puschel, James C. Hoe, and Jose M. F. Moura. 2018. SPIRAL: Extreme Performance Portability. *Proc. IEEE* 106, 11 (Nov. 2018), 1935–1968. https://doi.org/10.1109/JPROC.2018.2873289

[44] Mahdi Ghorbani, Mathieu Huot, Shideh Hashemian, and Amir Shaikhha. 2023. Compiling Structured Tensor Algebra. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (Oct. 2023), 229:204–229:233. https://doi.org/10.1145/3622804

[45] Solomon Golomb. 1966. Run-length encodings (corresp.). *IEEE transactions on information theory* 12, 3 (1966), 399–401. Publisher: Citeseer.

[46] Rafael C. Gonzalez and Richard E. Woods. 2006. *Digital Image Processing (3rd Edition)*. Prentice-Hall, Inc., USA.

[47] Tobias Grosser, Armin Groesslinger, and Christian Lengauer. 2012. Polly—performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters* 22, 04 (2012), 1250010.

[48] Gaël Guennebaud, Benoît Jacob, and others. 2010. Eigen v3. http://eigen.tuxfamily.org

[49] Bastian Hagedorn, Johannes Lenfers, Thomas Kœhler, Xueying Qin, Sergei Gorlatch, and Michel Steuwer. 2020. Achieving high-performance the functional way: a functional pearl on expressing high-performance optimizations as rewrite strategies. *Proceedings of the ACM on Programming Languages* 4, ICFP (Aug. 2020), 1–29. https://doi.org/10.1145/3408974

[50] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array programming with NumPy. *Nature* 585, 7825 (Sept. 2020), 357–362. https://doi.org/10.1038/s41586-020-2649-2 Number: 7825 Publisher: Nature Publishing Group.

[51] Rawn Henry, Olivia Hsu, Rohan Yadav, Stephen Chou, Kunle Olukotun, Saman Amarasinghe, and Fredrik Kjolstad. 2021. Compilation of sparse array programming models. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (Oct. 2021), 128:1–128:29. https://doi.org/10.1145/3485505

[52] Olivia Hsu, Maxwell Strange, Ritvik Sharma, Jaeyeon Won, Kunle Olukotun, Joel S. Emer, Mark A. Horowitz, and Fredrik Kjølstad. 2023. The Sparse Abstract Machine. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 710–726. https://doi.org/10.1145/3582016.3582051

[53] Yuanming Hu, Tzu-Mao Li, Luke Anderson, Jonathan Ragan-Kelley, and Frédo Durand. 2019. Taichi: a language for high-performance computation on spatially sparse data structures. *ACM Transactions on Graphics* 38, 6 (Nov. 2019), 201:1–201:16. https://doi.org/10.1145/3355089.3356506

[54] Dylan Hutchison, Bill Howe, and Dan Suciu. 2017. LaraDB: A Minimalist Kernel for Linear and Relational Algebra Computation. In *Proceedings of the 4th ACM SIGMOD Workshop on Algorithms and Systems for MapReduce and Beyond (BeyondMR'17)*. Association for Computing Machinery, New York, NY, USA, 1–10. https://doi.org/10.1145/3070607.3070608

[55] Yuka Ikarashi, Gilbert Louis Bernstein, Alex Reinking, Hasan Genc, and Jonathan Ragan-Kelley. 2022. Exocompilation for productive programming of hardware accelerators. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 703–718. https://doi.org/10.1145/3519939.3523446

[56] Eun-Jin Im and Katherine Yelick. 2001. Optimizing Sparse Matrix Computations for Register Reuse in SPARSITY. In *Computational Science — ICCS 2001 (Lecture Notes in Computer Science)*. Springer, Berlin, Heidelberg, 127–136. https://doi.org/10.1007/3-540-45545-0_22

[57] U Kang, Hanghang Tong, Jimeng Sun, Ching-Yung Lin, and Christos Faloutsos. 2011. GBASE: a scalable and general graph management system. In *Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1091–1099.

[58] Jeremy Kepner, Peter Aaltonen, David Bader, Aydin Buluç, Franz Franchetti, John Gilbert, Dylan Hutchison, Manoj Kumar, Andrew Lumsdaine, Henning Meyerhenke, Scott McMillan, Carl Yang, John D. Owens, Marcin Zalewski, Timothy Mattson, and Jose Moreira. 2016. Mathematical foundations of the GraphBLAS. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–9. https://doi.org/10.1109/HPEC.2016.7761646

[59] Fredrik Kjolstad, Willow Ahrens, Shoaib Kamil, and Saman Amarasinghe. 2019. Tensor Algebra Compilation with Workspaces. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO) (CGO)*. CGO, 180–192. https://doi.org/10.1109/CGO.2019.8661185

[60] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (Oct. 2017), 77:1–77:29. https://doi.org/10.1145/3133901

[61] Donald E. Knuth. 1997. *The Art of Computer Programming: Fundamental Algorithms, Volume 1.* Addison-Wesley Professional. Google-Books-ID: x9AsAwAAQBAJ.

[62] Vladimir Kotlyar, Keshav Pingali, and Paul Stodghill. 1997. A relational approach to the compilation of sparse matrix programs. In *Euro-Par'97 Parallel Processing (Lecture Notes in Computer Science)*, Christian Lengauer, Martin Griebl, and Sergei Gorlatch (Eds.). Springer, Berlin, Heidelberg, 318–327. https://doi.org/10.1007/BFb0002751

[63] Brenden M. Lake, Ruslan Salakhutdinov, and Joshua B. Tenenbaum. 2015. Human-level concept learning through probabilistic program induction. *Science* 350, 6266 (Dec. 2015), 1332–1338. https://doi.org/10.1126/science.aab3050 Publisher: American Association for the Advancement of Science.

[64] Daniel Langr and Pavel Tvrdík. 2016. Evaluation Criteria for Sparse Matrix Storage Formats. *IEEE Transactions on Parallel and Distributed Systems* 27, 2 (Feb. 2016), 428–440. https://doi.org/10.1109/TPDS.2015.2401575 Conference Name: IEEE Transactions on Parallel and Distributed Systems.

[65] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (Nov. 1998), 2278–2324. https://doi.org/10.1109/5.726791 Conference Name: Proceedings of the IEEE.

[66] Jure Leskovec, Kevin J. Lang, and Michael Mahoney. 2010. Empirical comparison of algorithms for network community detection. In *Proceedings of the 19th international conference on World wide web (WWW '10)*. Association for Computing Machinery, New York, NY, USA, 631–640. https://doi.org/10.1145/1772690.1772755

[67] Amanda Liu, Gilbert Louis Bernstein, Adam Chlipala, and Jonathan Ragan-Kelley. 2022. Verified tensor-program optimization via high-level scheduling rewrites. *Proceedings of the ACM on Programming Languages* 6, POPL (Jan. 2022), 55:1–55:28. https://doi.org/10.1145/3498717

[68] Weifeng Liu and Brian Vinter. 2015. CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication. In *Proceedings of the 29th ACM on International Conference on Supercomputing (ICS '15)*. Association for Computing Machinery, New York, NY, USA, 339–350. https://doi.org/10.1145/2751205.2751209

[69] Shangyu Luo, Zekai J Gao, Michael Gubanov, Luis L Perez, and Christopher Jermaine. 2018. Scalable linear algebra on a relational database system. *ACM SIGMOD Record* 47, 1 (2018), 24–31. Publisher: ACM New York, NY, USA.

[70] Tim Mattson, Timothy A Davis, Manoj Kumar, Aydin Buluc, Scott McMillan, José Moreira, and Carl Yang. 2019. LAGraph: A community effort to collect graph algorithms built on top of the GraphBLAS. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 276–284.

[71] Julian McAuley and Jure Leskovec. 2013. Hidden factors and hidden topics: understanding rating dimensions with review text. In *Proceedings of the 7th ACM conference on Recommender systems (RecSys '13)*. Association for Computing Machinery, New York, NY, USA, 165–172. https://doi.org/10.1145/2507157.2507163

[72] Tommy McMichen, Nathan Greiner, Peter Zhong, Federico Sossai, Atmn Patel, and Simone Campanoni. 2024. Representing Data Collections in an SSA Form. In *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, Edinburgh, United Kingdom, 308–321. https://doi.org/10.1109/CGO57630.2024.10444817

[73] Cleve Moler and Jack Little. 2020. A history of MATLAB. *Proceedings of the ACM on Programming Languages* 4, HOPL (June 2020), 81:1–81:67. https://doi.org/10.1145/3386331

[74] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-case Optimal Join Algorithms. *J. ACM* 65, 3 (March 2018), 16:1–16:40. https://doi.org/10.1145/3180143

[75] Dianne P O'Leary. 2009. *Scientific computing with case studies.* SIAM.

[76] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[77] Christos Psarras, Henrik Barthels, and Paolo Bientinesi. 2022. The Linear Algebra Mapping Problem. Current state of linear algebra languages and libraries. *ACM Trans. Math. Software* 48, 3 (Sept. 2022), 1–30. https://doi.org/10.1145/3549935 arXiv:1911.09421 [cs].

[78] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*. Association for Computing Machinery, New York, NY, USA, 519–530. https://doi.org/10.1145/2491956.2462176

[79] Jason Rumengan, Terry Yue Zhuo, and Conrad Sanderson. 2021. PyArmadillo: a streamlined linear algebra library for Python. *Journal of Open Source Software* 6, 66 (2021), 3051. https://doi.org/10.21105/joss.03051

[80] Yousef Saad. 2003. *Iterative methods for sparse linear systems* (2nd ed.). SIAM, Philadelphia.

[81] Maximilian Schleich, Amir Shaikhha, and Dan Suciu. 2023. Optimizing Tensor Programs on Flexible Storage. *Proceedings of the ACM on Management of Data* 1, 1 (May 2023), 37:1–37:27. https://doi.org/10.1145/3588717

[82] Ryan Senanayake, Changwan Hong, Ziheng Wang, Amalee Wilson, Stephen Chou, Shoaib Kamil, Saman Amarasinghe, and Fredrik Kjolstad. 2020. A sparse iteration space transformation framework for sparse tensor algebra. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (Nov. 2020), 158:1–158:30. https://doi.org/10.1145/3428226

[83] Amir Shaikhha, Mathieu Huot, Jaclyn Smith, and Dan Olteanu. 2022. Functional collection programming with semi-ring dictionaries. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (April 2022), 89:1–89:33. https://doi.org/10.1145/3527333

[84] Jia Shi. 2020. Column Partition and Permutation for Run Length Encoding in Columnar Databases. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 2873–2874. https://doi.org/10.1145/3318464.3384413

[85] Shaden Smith, Niranjay Ravindran, Nicholas D. Sidiropoulos, and George Karypis. 2015. SPLATT: Efficient and Parallel Sparse Tensor-Matrix Multiplication. In *Proceedings of the 2015 IEEE International Parallel and Distributed Processing Symposium (IPDPS '15)*. IEEE Computer Society, Washington, DC, USA, 61–70. https://doi.org/10.1109/IPDPS.2015.27

[86] Edgar Solomonik and Torsten Hoefler. 2015. Sparse Tensor Algebra as a Parallel Programming Model. *arXiv:1512.00066 [cs]* (Nov. 2015). http://arxiv.org/abs/1512.00066 arXiv: 1512.00066.

[87] Edgar Solomonik, Devin Matthews, Jeff Hammond, and James Demmel. 2013. Cyclops Tensor Framework: Reducing Communication and Eliminating Load Imbalance in Massively Parallel Contractions. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. 813–824. https://doi.org/10.1109/IPDPS.2013.112 ISSN: 1530-2075.

[88] Michael Stonebraker, Paul Brown, Donghui Zhang, and Jacek Becla. 2013. SciDB: A database management system for applications with complex analytics. *Computing in Science & Engineering* 15, 3 (2013), 54–62. Publisher: IEEE.

[89] Michelle Mills Strout, Mary Hall, and Catherine Olschanowsky. 2018. The Sparse Polyhedral Framework: Composing Compiler-Generated Inspector-Executor Code. *Proc. IEEE* 106, 11 (Nov. 2018), 1921–1934. https://doi.org/10.1109/JPROC.2018.2857721 Conference Name: Proceedings of the IEEE.

[90] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2020. Efficient Processing of Deep Neural Networks. *Synthesis Lectures on Computer Architecture* 15, 2 (June 2020), 1–341. https://doi.org/10.2200/S01004ED1V01Y202004CAC050 Publisher: Morgan & Claypool Publishers.

[91] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. *Proceedings of Machine Learning and Systems* 4 (April 2022), 302–315. https://proceedings.mlsys.org/paper_files/paper/2022/hash/c48e820389ae2420c1ad9d5856e1e41c-Abstract.html

[92] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. Torchsparse++: Efficient training and inference framework for sparse convolution on gpus. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 225–239.

[93] Odysseas G Tsatalos, Marvin H Solomon, and Yannis E Ioannidis. 1996. The GMAP: A versatile tool for physical data independence. *The VLDB Journal* 5 (1996), 101–118. Publisher: Springer.

[94] Field G. Van Zee and Robert A. van de Geijn. 2015. BLIS: A Framework for Rapidly Instantiating BLAS Functionality. *ACM Trans. Math. Softw.* 41, 3 (June 2015), 14:1–14:33. https://doi.org/10.1145/2764454

[95] Todd Veldhuizen. 2014. Triejoin: A Simple, Worst-Case Optimal Join Algorithm. https://doi.org/10.5441/002/ICDT.2014.13

[96] Sven Verdoolaege. 2010. ISL: An integer set library for the polyhedral model. In *International Congress on Mathematical Software*. Springer, 299–302.

[97] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, and Paul van Mulbregt. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17, 3 (March 2020), 261–272. https://doi.org/10.1038/s41592-019-0686-2 Bandiera_abtest: a Cc_license_type: cc_by Cg_type: Nature Research Journals Number: 3 Primary_atype: Reviews Publisher: Nature Publishing Group Subject_term: Biophysical chemistry;Computational biology and bioinformatics;Technology Subject_term_id: biophysical-chemistry;computational-biology-and-bioinformatics;technology.

[98] R. Vuduc, J.W. Demmel, K.A. Yelick, S. Kamil, R. Nishtala, and B. Lee. 2002. Performance Optimizations and Bounds for Sparse Matrix-Vector Multiply. In *SC '02: Proceedings of the 2002 ACM/IEEE Conference on Supercomputing*. 26–26. https://doi.org/10.1109/SC.2002.10025 ISSN: 1063-9535.

[99] Richard Vuduc, James W. Demmel, and Katherine A. Yelick. 2005. OSKI: A library of automatically tuned sparse matrix kernels. *Journal of Physics: Conference Series* 16 (Jan. 2005), 521–530. https://doi.org/10.1088/1742-6596/16/1/071 Publisher: IOP Publishing.

[100] Jaeyeon Won, Changwan Hong, Charith Mendis, Joel Emer, and Saman Amarasinghe. 2023. Unified Convolution Framework: A compiler-based approach to support sparse convolutions. *Proceedings of Machine Learning and Systems* 5 (March 2023), 666–679. https://proceedings.mlsys.org/paper_files/paper/2023/hash/ccf7262fb986e4367ccd3903960c57a0-Abstract-mlsys2023.html

[101] Rohan Yadav, Alex Aiken, and Fredrik Kjolstad. 2022. SpDISTAL: compiling distributed sparse tensor computations. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC '22)*. IEEE Press, Dallas, Texas, 1–15.

[102] Carl Yang, Aydın Buluç, and John D. Owens. 2018. Implementing Push-Pull Efficiently in GraphBLAS. In *Proceedings of the 47th International Conference on Parallel Processing (ICPP '18)*. Association for Computing Machinery, New York, NY, USA, 1–11. https://doi.org/10.1145/3225058.3225122

[103] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 660–678. https://doi.org/10.1145/3582016.3582047

[104] Tomofumi Yuki, Gautam Gupta, DaeGon Kim, Tanveer Pathan, and Sanjay Rajopadhye. 2012. Alphaz: A system for design space exploration in the polyhedral model. In *International Workshop on Languages and Compilers for Parallel Computing*. Springer, 17–31.

[105] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: leveraging Gustavson's algorithm to accelerate sparse matrix multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Virtual USA, 687–701. https://doi.org/10.1145/3445814.3446702

[106] Tuowen Zhao, Tobi Popoola, Mary Hall, Catherine Olschanowsky, and Michelle Strout. 2022. Polyhedral specification and code generation of sparse tensor contraction with co-iteration. *ACM Transactions on Architecture and Code Optimization* 20, 1 (2022), 1–26.

[107] Weijie Zhou, Yue Zhao, Xipeng Shen, and Wang Chen. 2020. Enabling Runtime SpMV Format Selection through an Overhead Conscious Method. *IEEE Transactions on Parallel and Distributed Systems* 31, 1 (Jan. 2020), 80–93. https://doi.org/10.1109/TPDS.2019.2932931 Conference Name: IEEE Transactions on Parallel and Distributed Systems.

A ARTIFACT APPENDIX

A.1 Abstract

In this artifact, we provide an archive of the Finch compiler at the time of writing and instructions to replicate all benchmarks in this paper. We note that the Finch compiler is separately available as open-source software. We claim that the results in this paper are reproducible with the provided artifact on an identical machine. Some results, especially those regarding MKL, may be architecture-specific.

A.2 Artifact check-list (meta-information)

Obligatory. Use just a few informal keywords in all fields applicable to your artifacts and remove the rest. This information is needed to find appropriate reviewers and gradually unify artifact meta information in Digital Libraries.

- **Algorithm:** Sparse Tensors, Compilers, Image Processing, Scientific Computing, Graph Analytics
- **Program:** Julia, C++, Python
- **Compilation:** Makefile, CMake, gcc, Julia, LLVM
- **Transformations:** Sparsity and Structural Specialization
- **Binary:** x86-64, also ARM
- **Data set:** Synthetic, SuiteSparse, MNIST, Omniglot, HumanSketches
- **Run-time environment:** Ubuntu 22.04.5 LTS, Linux 5.15.0-119-generic, Root Access
- **Hardware:** 12-core 2-socket Intel Xeon CPU E5-2680 v3 running at 2.50GHz with 128GB of memory.
- **Run-time state:** Sensitive to memory bandwidth, cache size
- **Execution:** Requires exclusive access to node for repeatable results
- **Metrics:** Execution time, Speedup
- **Output:** JSON table, PNG plot
- **Experiments:** SpMV, SpGEMM, Erosion, Histogram, Breadth-First Search, Shortest Path
- **How much disk space required (approximately)?:**
- **How much time is needed to prepare workflow (approximately)?:**
- **How much time is needed to complete experiments (approximately)?:**
- **Publicly available?:** yes
- **Code licenses (if publicly available)?:** MIT
- **Data licenses (if publicly available)?:** MIT
- **Workflow framework used?:** SLURM, Shell
- **Archived (provide DOI)?:** 10.5281/zenodo.14721701

A.3 Description

A.3.1 How delivered. The artifact may be downloaded from zeonodo at doi.org/10.5281/zenodo.14721701, or cloned from the `oopsla-25-artifact` branch of the FinchBenchmarks repository on GitHub at https://github.com/finch-tensor/FinchBenchmarks. The artifact contains a copy of the Finch.jl compiler version v1.1.0, and all benchmarks used in the paper. The artifact takes 1.6 MB to download, and 17 GB of disk space once it has been extracted and built and datasets have been downloaded and generated.

The `deps` subdirectory contains major dependencies required. The `spmv`, `spgemm`, `graphs`, and `images` directories contain the benchmarks corresponding to the SpMV, SpGEMM, Graphs, and Image Processing sections of the paper, respectively.

A.3.2 Hardware dependencies. This artifact was run on a 12-core 2-socket Intel Xeon CPU E5-2680 v3 running at 2.50GHz with 128GB of memory. The Intel MKL and CORA benchmarks require an x86-64 machine to build and run, but we believe that the other benchmarks can be built on ARM hardware.

A.3.3 Software dependencies. The results in the paper concern the following software dependencies, and special notes for building are included. Although we include the sources of the artifact, the artifact itself contains these repositories as submodules.

(1) Julia [18] v1.10.7 Julia can be installed via 'juliaup' at https://github.com/JuliaLang/juliaup or at https://julialang.org/downloads/.
(2) Finch 1.1.0 Finch is a registered Julia package and will be installed automatically during setup. However, if for whatever reason you would like to use the copy included with the artifact, you may run
```
julia --project=. -e 'using Pkg; Pkg.develop(PackageSpec(path="./deps/Finch.jl"))'
```
from the root of the artifact.
(3) TACO [60] at commit `1278503a1` from https://github.com/tensor-compiler/taco, corresponding to the "benchmark" branch. Taco requires CMake.
(4) Eigen 3.4.0 [48] from https://gitlab.com/libeigen/eigen.git
(5) GraphBLAS 9.4.2 from https://github.com/DrTimothyAldenDavis/GraphBLAS.
(6) LAGraph 1.1.4 from https://github.com/GraphBLAS/LAGraph.
(7) Graphs.jl 1.9 from https://github.com/JuliaGraphs/Graphs.jl.
(8) Intel MKL 2024.2 [1], available from https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl-download.html. This will need to be installed to the `deps/intel` folder.
(9) The Cora tensor compiler [39] at commit `8e7de1d7c` from https://github.com/pratikfegade/cora.git. An artifact is also available at https://doi.org/10.5281/zenodo.6326456. Cora requires MKL, LLVM 9.0.0, Z3 4.8.8, CMake, and Python 3. Instructions for building are available in the `cora/ae_appendix_supplement.pdf` file.

Our build process for all comparison frameworks is automated in our Makefile, included at the root of the directory. The `Project.toml` file contains all of the required Julia dependecies (as well as their versions). The `pyproject.toml` file contains all of the required python dependecies (as well as their versions).

We built our artifact on Ubuntu 22.04.5 LTS, Linux 5.15.0-119-generic, using the following dependencies:

(1) cmake 3.22.1
(2) gcc 11.4.0
(3) Python 3.10.12
(4) We used poetry 1.8.5 to manage python dependencies, which can be installed with `pip` or following https://python-poetry.org/docs/#installation.
(5) jq 1.6, git 2.34.1, curl 7.81.0, GNU tar 1.34, and UnZip 6.00

A.3.4 Data sets.

(1) SuiteSparse
The SuiteSparse Matrix Collection [33] is available at https://sparse.tamu.edu/. We use the MatrixDepot.jl package at https://github.com/JuliaLinearAlgebra/MatrixDepot.jl to download matrices from this collection. The precise datasets used for each benchmark are listed in the test harnesses.
(2) MNIST
The MNIST dataset [65] is available at http://yann.lecun.com/exdb/mnist/. We use the MLDatasets.jl package at https://github.com/JuliaML/MLDatasets.jl to download this dataset.
(3) Omniglot
The Omniglot dataset [63] is available at https://www.omniglot.com/. We use the MLDatasets.jl package at https://github.com/JuliaML/MLDatasets.jl to download this dataset.

(4) HumanSketches
We evaluate on a dataset of human line drawings [38], available at https://cybertron.cg.tu-berlin.de/eitz/projects/classifysketch/sketches_png.zip.
(5) Dip3masks
We also hand-selected a subset of mask images from a digital image processing textbook [46]. The precise set of images used is included with the artifact.
(6) Synthetic Data
Scripts are included to generate synthetic data. For spmv, we generate banded, triangular, and a reverse permutation matrix. For SpGEMM, we generate a series of increasingly larger uniformly random sparse matrices. For the Graphs dataset, we generate a few RMAT [26] graphs to match the dataset used by Yang et. al. [102].

A.4 Installation

A.4.1 1. Install system dependecies. Install Julia, Python, CMake, and other system dependencies as described in the software dependencies section.

A.4.2 2. Download the core dependencies. Several dependencies are included as submodules in the `deps/` folder, referencing the precise commits we used to build the artifact. Most of these can be installed via

```
git submodule update --init --recursive
```

MKL must be manually installed to the `deps/intel` folder. InstallIntelMKLversion2024.2,availablefromhttps://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl-download.html. You'll need to request an academic license on the website, then download an offline installer. There should be instructions on the website for how to run the install script. When asked, you can install to the `deps/intel` folder.

The makefile also contains instructions to clone the submodules.

A.4.3 3. Build the dependencies and benchmarks. The makefile contains targets to build all of the benchmarks and dependencies. We refer to each individual dependency for more detailed instructions. We expect that some system-specific adjustments to the makefile may be necessary to build CORA, as it has many complex subdependencies such as LLVM and Z3.

When all dependencies have been successfully installed, from the root of the artifact, run

```
make
```

A.4.4 4. Instantiate Runtime Environments. In this step, we will setup and install the Julia and Python environments. This can be achieved by running from the root of the artifact:

```
bash instantiate_environments.sh
```

which simply runs the following commands:

```
julia --project=. -e 'using Pkg; Pkg.instantiate(); Pkg.precompile()'
poetry install --no-root
```

A.5 Experiment workflow

A.5.1 1. Running the dataset generators. To generate the synthetic data used in the benchmarks, run

```
bash generate_data.sh
```

A.5.2 2. Running the benchmarks. To run all benchmarks, run

```
bash run_benchmarks.sh
```

This script will run all benchmarks and generate the JSON tables and PNG plots used in the paper.

A more fine-grained approach may be taken to run the benchmarks individually. Each experiment subdirectory contains a `run_*.sh` script that will run that particular benchmark, and the commands contained within may be modified to run different subsets of experiments. We expect that running the whole set of experiments may take in excess of 24 hours, so some experiments may be commented out. Additionally, if evaluators have access to a SLURM cluster, we include SLURM scripts that help accelerate the process, which may need adaptation to your particular cluster. It is convenient to use `jq` to combine json outputs from parallel runs, for example,

```
jq -s 'add' results_*.json > combined_results.json
```

A.5.3 3. Plotting the output. To generate the plots used for each experiment, run the corresponding chart.py script in each experiment directory. For example, to generate the plots for the SpMV experiment, run from within the `spmv` directory

```
poetry run python chart.py
```

Plots will be generated in the corresponding `charts/` directory of each experiment.

A.6 Evaluation and expected result

Reference JSON and PNG plot results are stored for each experiement with the prefix `reference_`. Evaluators can compare the reference plots with the result plots. To generate our geomean speedup claims, evaluators may run

```
poetry run get_geomean.py
```

in the corresponding experiment directory and compare to the text.

A.7 Reusability Guide

All of the julia run scripts support a `--help` flag describing parameters which allowing one to customize the experiments. Separately, the Finch compiler is a featureful Julia package, and the evaluators are encouraged to experiment with the compiler using the documentation at https://finch-tensor.github.io/Finch.jl/stable/ and https://finch-tensor.github.io/Finch.jl/stable/docs/language/calling_finch/. Reviewers may also build the documentation locally by running `julia docs/make.jl` from the root of the `Finch.jl` directory. Some example Finch programs are given in the `docs/examples` directory.

B GRAPH ALGORITHM LISTINGS

B.1 Finch Breadth-First Search

```
    function bfs_finch_kernel(edges, edgesT, source=5, alpha = 0.01)
    (n, m) = size(edges)
    edges = pattern!(edges)
    @assert n == m
    F = Tensor(SparseByteMap(Pattern()), n)
    _F = Tensor(SparseByteMap(Pattern()), n)
    @finch F[source] = true
    F_nnz = 1
    V = Tensor(Dense(Element(false)), n)
    @finch V[source] = true
    P = Tensor(Dense(Element(0)), n)
    @finch P[source] = source
    while F_nnz > 0
        if F_nnz/m > alpha # pull
            p = ShortCircuitScalar{0}()
            _F .= false
            for k=_
                if !V[k]
                    p .= 0
                    for j=_
                        if F[follow(j)] && AT[j, k]
                            p[] <<choose(0)>>= j
                        end
                    end
                    if p[] != 0
                        _F[k] |= true
                        P[k] = p[]
                    end
                end
            end
        else # push
            _F .= false
            for j=_, k=_
                if F[j] && A[k, j] && !(V[k])
                    _F[k] |= true
                    P[k] <<choose(0)>>= j
                end
            end
        end
        c = Scalar(0)
        @finch begin
            for k=_
                let _f = _F[k]
                    V[k] |= _f
                    c[] += _f
                end
            end
        end
        (F, _F) = (_F, F)
        F_nnz = c[]
    end
    return P
end
```

B.2 GraphBLAS Breadth-First Search

```
//------------------------------------------------------------------------------
// LAGr_BreadthFirstSearch:  breadth-first search dispatch
//------------------------------------------------------------------------------

// LAGraph, (c) 2019-2022 by The LAGraph Contributors, All Rights Reserved.
// SPDX-License-Identifier: BSD-2-Clause
//
// For additional details (including references to third party source code and
// other files) see the LICENSE file or contact permission@sei.cmu.edu. See
// Contributors.txt for a full list of contributors. Created, in part, with
// funding and support from the U.S. Government (see Acknowledgments.txt file).
// DM22-0790

// Contributed by Scott McMillan, SEI Carnegie Mellon University

//------------------------------------------------------------------------------

// Breadth-first-search via push/pull method if using SuiteSparse:GraphBLAS
// and its GxB extensions, or a push-only method otherwise.  The former is
// much faster.
```

```
// This is an Advanced algorithm.  SuiteSparse can use a push/pull method if
// G->AT and G->out_degree are provided.  G->AT is not required if G is
// undirected.  The vanilla method is always push-only.

#include "LG_alg_internal.h"

int LAGr_BreadthFirstSearch
(
    // output:
    GrB_Vector *level,
    GrB_Vector *parent,
    // input:
    const LAGraph_Graph G,
    GrB_Index src,
    char *msg
)
{

#if LAGRAPH_SUITESPARSE
    return LG_BreadthFirstSearch_SSGrB   (level, parent, G, src, msg) ;
#else
    return LG_BreadthFirstSearch_vanilla (level, parent, G, src, msg) ;
#endif
}

//------------------------------------------------------------------------------
// LG_BreadthFirstSearch_SSGrB:  BFS using Suitesparse extensions
//------------------------------------------------------------------------------

// LAGraph, (c) 2019-2022 by The LAGraph Contributors, All Rights Reserved.
// SPDX-License-Identifier: BSD-2-Clause
//
// For additional details (including references to third party source code and
// other files) see the LICENSE file or contact permission@sei.cmu.edu. See
// Contributors.txt for a full list of contributors. Created, in part, with
// funding and support from the U.S. Government (see Acknowledgments.txt file).
// DM22-0790

// Contributed by Timothy A. Davis, Texas A&M University

//------------------------------------------------------------------------------

// This is an Advanced algorithm.  G->AT and G->out_degree are required for
// this method to use push-pull optimization.  If not provided, this method
// defaults to a push-only algorithm, which can be slower.  This is not
// user-callable (see LAGr_BreadthFirstSearch instead).  G->AT and
// G->out_degree are not computed if not present.

// References:
//
// Carl Yang, Aydin Buluc, and John D. Owens. 2018. Implementing Push-Pull
// Efficiently in GraphBLAS. In Proceedings of the 47th International
// Conference on Parallel Processing (ICPP 2018). ACM, New York, NY, USA,
// Article 89, 11 pages. DOI: https://doi.org/10.1145/3225058.3225122
//
// Scott Beamer, Krste Asanovic and David A. Patterson, The GAP Benchmark
// Suite, http://arxiv.org/abs/1508.03619, 2015.  http://gap.cs.berkeley.edu/

// revised by Tim Davis (davis@tamu.edu), Texas A&M University

#define LG_FREE_WORK        \
{                           \
    GrB_free (&w) ;         \
    GrB_free (&q) ;         \
}

#define LG_FREE_ALL         \
{                           \
    LG_FREE_WORK ;          \
    GrB_free (&pi) ;        \
    GrB_free (&v) ;         \
}

#include "LG_internal.h"

int LG_BreadthFirstSearch_SSGrB
(
    GrB_Vector *level,
    GrB_Vector *parent,
    const LAGraph_Graph G,
    GrB_Index src,

```

```
    char *msg
)
{

    //--------------------------------------------------------------------------
    // check inputs
    //--------------------------------------------------------------------------

    LG_CLEAR_MSG ;
    GrB_Vector q = NULL ;           // the current frontier
    GrB_Vector w = NULL ;           // to compute work remaining
    GrB_Vector pi = NULL ;          // parent vector
    GrB_Vector v = NULL ;           // level vector

#if !LAGRAPH_SUITESPARSE
    LG_ASSERT (false, GrB_NOT_IMPLEMENTED) ;
#else

    bool compute_level  = (level != NULL) ;
    bool compute_parent = (parent != NULL) ;
    if (compute_level ) (*level ) = NULL ;
    if (compute_parent) (*parent) = NULL ;
    LG_ASSERT_MSG (compute_level || compute_parent, GrB_NULL_POINTER,
        "either level or parent must be non-NULL") ;

    LG_TRY (LAGraph_CheckGraph (G, msg)) ;

    //--------------------------------------------------------------------------
    // get the problem size and cached properties
    //--------------------------------------------------------------------------

    GrB_Matrix A = G->A ;

    GrB_Index n, nvals ;
    GRB_TRY (GrB_Matrix_nrows (&n, A)) ;
    LG_ASSERT_MSG (src < n, GrB_INVALID_INDEX, "invalid source node") ;

    GRB_TRY (GrB_Matrix_nvals (&nvals, A)) ;

    GrB_Matrix AT = NULL ;
    GrB_Vector Degree = G->out_degree ;
    if (G->kind == LAGraph_ADJACENCY_UNDIRECTED ||
       (G->kind == LAGraph_ADJACENCY_DIRECTED &&
        G->is_symmetric_structure == LAGraph_TRUE))
    {
        // AT and A have the same structure and can be used in both directions
        AT = G->A ;
    }
    else
    {
        // AT = A' is different from A.  If G->AT is NULL, then a push-only
        // method is used.
        AT = G->AT ;
    }

    // direction-optimization requires G->AT (if G is directed) and
    // G->out_degree (for both undirected and directed cases)
    bool push_pull = (Degree != NULL && AT != NULL) ;

    // determine the semiring type
    GrB_Type int_type = (n > INT32_MAX) ? GrB_INT64 : GrB_INT32 ;
    GrB_Semiring semiring ;

    if (compute_parent)
    {
        // use the ANY_SECONDI_INT* semiring: either 32 or 64-bit depending on
        // the # of nodes in the graph.
        semiring = (n > INT32_MAX) ?
            GxB_ANY_SECONDI_INT64 : GxB_ANY_SECONDI_INT32 ;

        // create the parent vector.  pi(i) is the parent id of node i
        GRB_TRY (GrB_Vector_new (&pi, int_type, n)) ;
        GRB_TRY (GxB_set (pi, GxB_SPARSITY_CONTROL, GxB_BITMAP + GxB_FULL)) ;
        // pi (src) = src denotes the root of the BFS tree
        GRB_TRY (GrB_Vector_setElement (pi, src, src)) ;

        // create a sparse integer vector q, and set q(src) = src
        GRB_TRY (GrB_Vector_new (&q, int_type, n)) ;
        GRB_TRY (GrB_Vector_setElement (q, src, src)) ;
    }
    else
```

```
    {
        // only the level is needed, use the LAGraph_any_one_bool semiring
        semiring = LAGraph_any_one_bool ;

        // create a sparse boolean vector q, and set q(src) = true
        GRB_TRY (GrB_Vector_new (&q, GrB_BOOL, n)) ;
        GRB_TRY (GrB_Vector_setElement (q, true, src)) ;
    }

    if (compute_level)
    {
        // create the level vector. v(i) is the level of node i
        // v (src) = 0 denotes the source node
        GRB_TRY (GrB_Vector_new (&v, int_type, n)) ;
        GRB_TRY (GxB_set (v, GxB_SPARSITY_CONTROL, GxB_BITMAP + GxB_FULL)) ;
        GRB_TRY (GrB_Vector_setElement (v, 0, src)) ;
    }

    // workspace for computing work remaining
    GRB_TRY (GrB_Vector_new (&w, GrB_INT64, n)) ;

    GrB_Index nq = 1 ;          // number of nodes in the current level
    double alpha = 8.0 ;
    double beta1 = 8.0 ;
    double beta2 = 512.0 ;
    int64_t n_over_beta1 = (int64_t) (((double) n) / beta1) ;
    int64_t n_over_beta2 = (int64_t) (((double) n) / beta2) ;

    //------------------------------------------------------------------------------
    // BFS traversal and label the nodes
    //------------------------------------------------------------------------------

    bool do_push = true ;       // start with push
    GrB_Index last_nq = 0 ;
    int64_t edges_unexplored = nvals ;
    bool any_pull = false ;     // true if any pull phase has been done

    // {!mask} is the set of unvisited nodes
    GrB_Vector mask = (compute_parent) ? pi : v ;

    for (int64_t nvisited = 1, k = 1 ; nvisited < n ; nvisited += nq, k++)
    {

        //--------------------------------------------------------------------------
        // select push vs pull
        //--------------------------------------------------------------------------

        if (push_pull)
        {
            if (do_push)
            {
                // check for switch from push to pull
                bool growing = nq > last_nq ;
                bool switch_to_pull = false ;
                if (edges_unexplored < n)
                {
                    // very little of the graph is left; disable the pull
                    push_pull = false ;
                }
                else if (any_pull)
                {
                    // once any pull phase has been done, the # of edges in the
                    // frontier has no longer been tracked.  But now the BFS
                    // has switched back to push, and we're checking for yet
                    // another switch to pull.  This switch is unlikely, so
                    // just keep track of the size of the frontier, and switch
                    // if it starts growing again and is getting big.
                    switch_to_pull = (growing && nq > n_over_beta1) ;
                }
                else
                {
                    // update the # of unexplored edges
                    // w<q>=Degree
                    // w(i) = outdegree of node i if node i is in the queue
                    GRB_TRY (GrB_assign (w, q, NULL, Degree, GrB_ALL, n,
                        GrB_DESC_RS)) ;
                    // edges_in_frontier = sum (w) = # of edges incident on all
                    // nodes in the current frontier
                    int64_t edges_in_frontier = 0 ;
                    GRB_TRY (GrB_reduce (&edges_in_frontier, NULL,
                        GrB_PLUS_MONOID_INT64, w, NULL)) ;
```

```
                    edges_unexplored -= edges_in_frontier ;
                    switch_to_pull = growing &&
                        (edges_in_frontier > (edges_unexplored / alpha)) ;
                }
                if (switch_to_pull)
                {
                    // switch from push to pull
                    do_push = false ;
                }
            }
            else
            {
                // check for switch from pull to push
                bool shrinking = nq < last_nq ;
                if (shrinking && (nq <= n_over_beta2))
                {
                    // switch from pull to push
                    do_push = true ;
                }
            }
            any_pull = any_pull || (!do_push) ;
        }

        //----------------------------------------------------------------------
        // q = kth level of the BFS
        //----------------------------------------------------------------------

        int sparsity = do_push ? GxB_SPARSE : GxB_BITMAP ;
        GRB_TRY (GxB_set (q, GxB_SPARSITY_CONTROL, sparsity)) ;

        // mask is pi if computing parent, v if computing just level
        if (do_push)
        {
            // push (saxpy-based vxm):  q'{!mask} = q'*A
            GRB_TRY (GrB_vxm (q, mask, NULL, semiring, q, A, GrB_DESC_RSC)) ;
        }
        else
        {
            // pull (dot-product-based mxv):  q{!mask} = AT*q
            GRB_TRY (GrB_mxv (q, mask, NULL, semiring, AT, q, GrB_DESC_RSC)) ;
        }

        //----------------------------------------------------------------------
        // done if q is empty
        //----------------------------------------------------------------------

        last_nq = nq ;
        GRB_TRY (GrB_Vector_nvals (&nq, q)) ;
        if (nq == 0)
        {
            break ;
        }

        //----------------------------------------------------------------------
        // assign parents/levels
        //----------------------------------------------------------------------

        if (compute_parent)
        {
            // q(i) currently contains the parent id of node i in tree.
            // pi{q} = q
            GRB_TRY (GrB_assign (pi, q, NULL, q, GrB_ALL, n, GrB_DESC_S)) ;
        }
        if (compute_level)
        {
            // v{q} = k, the kth level of the BFS
            GRB_TRY (GrB_assign (v, q, NULL, k, GrB_ALL, n, GrB_DESC_S)) ;
        }
    }

    //--------------------------------------------------------------------------
    // free workspace and return result
    //--------------------------------------------------------------------------

    if (compute_parent) (*parent) = pi ;
    if (compute_level ) (*level ) = v ;
    LG_FREE_WORK ;
    return (GrB_SUCCESS) ;
#endif
}
```

B.3 Finch Bellman-Ford

```
function bellmanford_finch_kernel(edges, source=1)
    (n, m) = size(edges)
    @assert n == m
    dists_prev = Tensor(Dense(Element(Inf)), n)
    dists_prev[source] = 0
    dists = Tensor(Dense(Element(Inf)), n)
    active_prev = Tensor(SparseByteMap(Pattern()), n)
    active_prev[source] = true
    active = Tensor(SparseByteMap(Pattern()), n)
    parents = Tensor(Dense(Element(0)), n)
    for iter = 1:n
        @finch begin
            for j=_
                if active_prev[j]
                    dists[j] <<min>>= dists_prev[j]
                end
            end
        end
        @finch begin
            active .= false
            for j = _
                if active_prev[j]
                    for i = _
                        let d = dists_prev[j] + edges[i, j]
                            dists[i] <<min>>= d
                            active[i] |= d < dists_prev[i]
                        end
                    end
                end
            end
        end
        if countstored(active) == 0
            break
        end
        dists_prev, dists = dists, dists_prev
        active_prev, active = active, active_prev
    end
    @finch begin
        for j = _
            for i = _
                let d = edges[i, j]
                    if d < Inf && dists[j] + d <= dists[i]
                        parents[i] <<choose(0)>>= j
                    end
                end
            end
        end
    end
    return (dists=dists, parents=parents)
end
```

B.4 GraphBLAS Bellman-Ford

```
//------------------------------------------------------------------------------
// LAGraph_BF_full1a.c: Bellman-Ford single-source shortest paths, returns tree,
// while diagonal of input matrix A needs not to be explicit 0
//------------------------------------------------------------------------------

// LAGraph, (c) 2019-2022 by The LAGraph Contributors, All Rights Reserved.
// SPDX-License-Identifier: BSD-2-Clause
//
// For additional details (including references to third party source code and
// other files) see the LICENSE file or contact permission@sei.cmu.edu. See
// Contributors.txt for a full list of contributors. Created, in part, with
// funding and support from the U.S. Government (see Acknowledgments.txt file).
// DM22-0790

// Contributed by Jinhao Chen and Timothy A. Davis, Texas A&M University

//------------------------------------------------------------------------------

// This is the fastest variant that computes both the parent & the path length.

// LAGraph_BF_full1a: Bellman-Ford single source shortest paths, returning both
// the path lengths and the shortest-path tree.

// LAGraph_BF_full performs a Bellman-Ford to find out shortest path, parent
// nodes along the path and the hops (number of edges) in the path from given
// source vertex s in the range of [0, n) on graph given as matrix A with size
```

```
// n*n. The sparse matrix A has entry A(i, j) if there is an edge from vertex i
// to vertex j with weight w, then A(i, j) = w.

// LAGraph_BF_full1a returns GrB_SUCCESS if it succeeds.  In this case, there
// are no negative-weight cycles in the graph, and d, pi, and h are returned.
// The vector d has d(k) as the shortest distance from s to k. pi(k) = p+1,
// where p is the parent node of k-th node in the shortest path. In particular,
// pi(s) = 0. h(k) = hop(s, k), the number of edges from s to k in the shortest
// path.

// If the graph has a negative-weight cycle, GrB_NO_VALUE is returned, and the
// GrB_Vectors d(k), pi(k) and h(k)  (i.e., *pd_output, *ppi_output and
// *ph_output respectively) will be NULL when negative-weight cycle detected.

// Otherwise, other errors such as GrB_OUT_OF_MEMORY, GrB_INVALID_OBJECT, and
// so on, can be returned, if these errors are found by the underlying
// GrB_* functions.

//------------------------------------------------------------------------------

#define LG_FREE_WORK                    \
{                                       \
    GrB_free(&d);                       \
    GrB_free(&dmasked);                 \
    GrB_free(&dless);                   \
    GrB_free(&Atmp);                    \
    GrB_free(&BF_Tuple3);               \
    GrB_free(&BF_lMIN_Tuple3);          \
    GrB_free(&BF_PLUSrhs_Tuple3);       \
    GrB_free(&BF_LT_Tuple3);            \
    GrB_free(&BF_lMIN_Tuple3_Monoid);   \
    GrB_free(&BF_lMIN_PLUSrhs_Tuple3); \
    LAGraph_Free ((void**)&I, NULL);    \
    LAGraph_Free ((void**)&J, NULL);    \
    LAGraph_Free ((void**)&w, NULL);    \
    LAGraph_Free ((void**)&W, NULL);    \
    LAGraph_Free ((void**)&h, NULL);    \
    LAGraph_Free ((void**)&pi, NULL);   \
}

#define LG_FREE_ALL                     \
{                                       \
    LG_FREE_WORK ;                      \
    GrB_free (pd_output);               \
    GrB_free (ppi_output);              \
    GrB_free (ph_output);               \
}

#include <LAGraph.h>
#include <LAGraphX.h>
#include <LG_internal.h>  // from src/utility

typedef void (*LAGraph_binary_function) (void *, const void *, const void *) ;

//------------------------------------------------------------------------------
// data type for each entry of the adjacent matrix A and "distance" vector d;
// <INFINITY,INFINITY,INFINITY> corresponds to nonexistence of a path, and
// the value  <0, 0, NULL> corresponds to a path from a vertex to itself
//------------------------------------------------------------------------------

typedef struct
{
    double w;    // w  corresponds to a path weight.
    GrB_Index h; // h  corresponds to a path size or number of hops.
    GrB_Index pi;// pi corresponds to the penultimate vertex along a path.
                 // vertex indexed as 1, 2, 3, ... , V, and pi = 0 (as nil)
                 // for u=v, and pi = UINT64_MAX (as inf) for (u,v) not in E
}
BF_Tuple3_struct;

//------------------------------------------------------------------------------
// binary functions, z=f(x,y), where Tuple3xTuple3 -> Tuple3
//------------------------------------------------------------------------------

void BF_lMIN3
(
    BF_Tuple3_struct *z,
    const BF_Tuple3_struct *x,
    const BF_Tuple3_struct *y
)
{
```

```
    if (x->w < y->w
        || (x->w == y->w && x->h < y->h)
        || (x->w == y->w && x->h == y->h && x->pi < y->pi))
    {
        if (z != x) { *z = *x; }
    }
    else
    {
        *z = *y;
    }
}

void BF_PLUSrhs3
(
    BF_Tuple3_struct *z,
    const BF_Tuple3_struct *x,
    const BF_Tuple3_struct *y
)
{
    z->w = x->w + y->w ;
    z->h = x->h + y->h ;
    z->pi = (x->pi != UINT64_MAX && y->pi != 0) ?  y->pi : x->pi ;
}

void BF_LT3
(
    bool *z,
    const BF_Tuple3_struct *x,
    const BF_Tuple3_struct *y
)
{
    (*z) = (x->w < y->w
        || (x->w == y->w && x->h < y->h)
        || (x->w == y->w && x->h == y->h && x->pi < y->pi)) ;
}

// Given a n-by-n adjacency matrix A and a source vertex s.
// If there is no negative-weight cycle reachable from s, return the distances
// of shortest paths from s and parents along the paths as vector d. Otherwise,
// returns d=NULL if there is a negtive-weight cycle.
// pd_output is pointer to a GrB_Vector, where the i-th entry is d(s,i), the
//   sum of edges length in the shortest path
// ppi_output is pointer to a GrB_Vector, where the i-th entry is pi(i), the
//   parent of i-th vertex in the shortest path
// ph_output is pointer to a GrB_Vector, where the i-th entry is h(s,i), the
//   number of edges from s to i in the shortest path
// A has weights on corresponding entries of edges
// s is given index for source vertex
GrB_Info LAGraph_BF_full1a
(
    GrB_Vector *pd_output,      //the pointer to the vector of distance
    GrB_Vector *ppi_output,     //the pointer to the vector of parent
    GrB_Vector *ph_output,      //the pointer to the vector of hops
    const GrB_Matrix A,         //matrix for the graph
    const GrB_Index s           //given index of the source
)
{
    GrB_Info info;
    char *msg = NULL ;
    // tmp vector to store distance vector after n (i.e., V) loops
    GrB_Vector d = NULL, dmasked = NULL, dless = NULL;
    GrB_Matrix Atmp = NULL;
    GrB_Type BF_Tuple3;

    GrB_BinaryOp BF_lMIN_Tuple3;
    GrB_BinaryOp BF_PLUSrhs_Tuple3;
    GrB_BinaryOp BF_LT_Tuple3;

    GrB_Monoid BF_lMIN_Tuple3_Monoid;
    GrB_Semiring BF_lMIN_PLUSrhs_Tuple3;

    GrB_Index nrows, ncols, n, nz;  // n = # of row/col, nz = # of nnz in graph
    GrB_Index *I = NULL, *J = NULL; // for col/row indices of entries from A
    GrB_Index *h = NULL, *pi = NULL;
    double *w = NULL;
    BF_Tuple3_struct *W = NULL;

    if (pd_output  != NULL) *pd_output  = NULL;
    if (ppi_output != NULL) *ppi_output = NULL;
    if (ph_output  != NULL) *ph_output  = NULL;
```

```
    LG_ASSERT (A != NULL && pd_output != NULL &&
        ppi_output != NULL && ph_output != NULL, GrB_NULL_POINTER) ;

    GRB_TRY (GrB_Matrix_nrows (&nrows, A)) ;
    GRB_TRY (GrB_Matrix_ncols (&ncols, A)) ;
    GRB_TRY (GrB_Matrix_nvals (&nz, A));
    LG_ASSERT_MSG (nrows == ncols, -1002, "A must be square") ;
    n = nrows;
    LG_ASSERT_MSG (s < n, GrB_INVALID_INDEX, "invalid source node") ;

    //--------------------------------------------------------------------------
    // create all GrB_Type GrB_BinaryOp GrB_Monoid and GrB_Semiring
    //--------------------------------------------------------------------------
    // GrB_Type
    GRB_TRY (GrB_Type_new(&BF_Tuple3, sizeof(BF_Tuple3_struct)));

    // GrB_BinaryOp
    GRB_TRY (GrB_BinaryOp_new(&BF_LT_Tuple3,
        (LAGraph_binary_function) (&BF_LT3), GrB_BOOL, BF_Tuple3, BF_Tuple3));
    GRB_TRY (GrB_BinaryOp_new(&BF_lMIN_Tuple3,
        (LAGraph_binary_function) (&BF_lMIN3), BF_Tuple3, BF_Tuple3,BF_Tuple3));
    GRB_TRY (GrB_BinaryOp_new(&BF_PLUSrhs_Tuple3,
        (LAGraph_binary_function)(&BF_PLUSrhs3),
        BF_Tuple3, BF_Tuple3, BF_Tuple3));

    // GrB_Monoid
    BF_Tuple3_struct BF_identity = (BF_Tuple3_struct) { .w = INFINITY,
        .h = UINT64_MAX, .pi = UINT64_MAX };
    GRB_TRY (GrB_Monoid_new_UDT(&BF_lMIN_Tuple3_Monoid, BF_lMIN_Tuple3,
        &BF_identity));

    //GrB_Semiring
    GRB_TRY (GrB_Semiring_new(&BF_lMIN_PLUSrhs_Tuple3,
        BF_lMIN_Tuple3_Monoid, BF_PLUSrhs_Tuple3));

    //--------------------------------------------------------------------------
    // allocate arrays used for tuplets
    //--------------------------------------------------------------------------
#if 1
    LAGRAPH_TRY (LAGraph_Malloc ((void **) &I, nz, sizeof(GrB_Index), msg)) ;
    LAGRAPH_TRY (LAGraph_Malloc ((void **) &J, nz, sizeof(GrB_Index), msg)) ;
    LAGRAPH_TRY (LAGraph_Malloc ((void **) &w, nz, sizeof(double), msg)) ;
    LAGRAPH_TRY (LAGraph_Malloc ((void **) &W, nz, sizeof(BF_Tuple3_struct),
        msg)) ;

    //--------------------------------------------------------------------------
    // create matrix Atmp based on A, while its entries become BF_Tuple3 type
    //--------------------------------------------------------------------------

    GRB_TRY (GrB_Matrix_extractTuples_FP64(I, J, w, &nz, A));
    int nthreads, nthreads_outer, nthreads_inner ;
    LG_TRY (LAGraph_GetNumThreads (&nthreads_outer, &nthreads_inner, msg)) ;
    nthreads = nthreads_outer * nthreads_inner ;
    printf ("nthreads %d\n", nthreads) ;
    int64_t k;
    #pragma omp parallel for num_threads(nthreads) schedule(static)
    for (k = 0; k < nz; k++)
    {
        W[k] = (BF_Tuple3_struct) { .w = w[k], .h = 1, .pi = I[k] + 1 };
    }
    GRB_TRY (GrB_Matrix_new(&Atmp, BF_Tuple3, n, n));
    GRB_TRY (GrB_Matrix_build_UDT(Atmp, I, J, W, nz, BF_lMIN_Tuple3));
    LAGraph_Free ((void**)&I, NULL);
    LAGraph_Free ((void**)&J, NULL);
    LAGraph_Free ((void**)&W, NULL);
    LAGraph_Free ((void**)&w, NULL);

#else

    todo: GraphBLAS could use a new kind of unary operator, not z=f(x), but

    [z,flag] = f (aij, i, j, k, nrows, ncols, nvals, etc, ...)
    flag: keep or discard.  Combines GrB_apply and GxB_select.

    builtins:
        f(...) =
            i, bool is true
            j, bool is true
            i+j*nrows, etc.
            k
            tril, triu (like GxB_select): return aij, and true/false boolean
```

```
        z=f(x,i).  x: double, z:tuple3, i:GrB_Index with the row index of x
        // z = (BF_Tuple3_struct) { .w = x, .h = 1, .pi = i + 1 };

    GrB_apply (Atmp, op, A, ...)

    in the BFS, this is used:
        op:  z = f ( .... ) = i
        to replace x(i) with i

#endif

    //------------------------------------------------------------------------
    // create and initialize "distance" vector d, dmasked and dless
    //------------------------------------------------------------------------
    GRB_TRY (GrB_Vector_new(&d, BF_Tuple3, n));
    // make d dense
    GRB_TRY (GrB_Vector_assign_UDT(d, NULL, NULL, (void*)&BF_identity,
        GrB_ALL, n, NULL));
    // initial distance from s to itself
    BF_Tuple3_struct d0 = (BF_Tuple3_struct) { .w = 0, .h = 0, .pi = 0 };
    GRB_TRY (GrB_Vector_setElement_UDT(d, &d0, s));

    // creat dmasked as a sparse vector with only one entry at s
    GRB_TRY (GrB_Vector_new(&dmasked, BF_Tuple3, n));
    GRB_TRY (GrB_Vector_setElement_UDT(dmasked, &d0, s));

    // create dless
    GRB_TRY (GrB_Vector_new(&dless, GrB_BOOL, n));

    //------------------------------------------------------------------------
    // start the Bellman Ford process
    //------------------------------------------------------------------------
    bool any_dless= true;      // if there is any newly found shortest path
    int64_t iter = 0;          // number of iterations

    // terminate when no new path is found or more than V-1 loops
    while (any_dless && iter < n - 1)
    {
        // execute semiring on dmasked and A, and save the result to dmasked
        GRB_TRY (GrB_vxm(dmasked, GrB_NULL, GrB_NULL,
            BF_lMIN_PLUSrhs_Tuple3, dmasked, Atmp, GrB_NULL));

        // dless = d .< dtmp
        GRB_TRY (GrB_eWiseMult(dless, NULL, NULL, BF_LT_Tuple3, dmasked, d,
            NULL));

        // if there is no entry with smaller distance then all shortest paths
        // are found
        GRB_TRY (GrB_reduce (&any_dless, NULL, GrB_LOR_MONOID_BOOL, dless,
            NULL)) ;
        if(any_dless)
        {
            // update all entries with smaller distances
            //GRB_TRY (GrB_apply(d, dless, NULL, BF_Identity_Tuple3,
            //    dmasked, NULL));
            GRB_TRY (GrB_assign(d, dless, NULL, dmasked, GrB_ALL, n, NULL));

            // only use entries that were just updated
            //GRB_TRY (GrB_Vector_clear(dmasked));
            //GRB_TRY (GrB_apply(dmasked, dless, NULL, BF_Identity_Tuple3,
            //    d, NULL));
            //try:
            GRB_TRY (GrB_assign(dmasked, dless, NULL, d, GrB_ALL, n, GrB_DESC_R));
        }
        iter ++;
    }

    // check for negative-weight cycle only when there was a new path in the
    // last loop, otherwise, there can't be a negative-weight cycle.
    if (any_dless)
    {
        // execute semiring again to check for negative-weight cycle
        GRB_TRY (GrB_vxm(dmasked, GrB_NULL, GrB_NULL,
            BF_lMIN_PLUSrhs_Tuple3, dmasked, Atmp, GrB_NULL));

        // dless = d .< dtmp
        GRB_TRY (GrB_eWiseMult(dless, NULL, NULL, BF_LT_Tuple3, dmasked, d,
            NULL));

        // if there is no entry with smaller distance then all shortest paths
```

```
        // are found
        GRB_TRY (GrB_reduce (&any_dless, NULL, GrB_LOR_MONOID_BOOL, dless,
            NULL)) ;
        if(any_dless)
        {
            // printf("A negative-weight cycle found. \n");
            LG_FREE_ALL;
            return (GrB_NO_VALUE) ;
        }
    }

    //------------------------------------------------------------------------
    // extract tuple from "distance" vector d and create GrB_Vectors for output
    //------------------------------------------------------------------------

    LAGRAPH_TRY (LAGraph_Malloc ((void **) &I, n, sizeof(GrB_Index), msg)) ;
    LAGRAPH_TRY (LAGraph_Malloc ((void **) &W, n, sizeof(BF_Tuple3_struct),
        msg)) ;
    LAGRAPH_TRY (LAGraph_Malloc ((void **) &w, n, sizeof(double), msg)) ;
    LAGRAPH_TRY (LAGraph_Malloc ((void **) &h, n, sizeof(GrB_Index), msg)) ;
    LAGRAPH_TRY (LAGraph_Malloc ((void **) &pi, n, sizeof(GrB_Index), msg)) ;

    // todo: create 3 unary ops, and use GrB_apply?

    GRB_TRY (GrB_Vector_extractTuples_UDT (I, (void *) W, &n, d));

    for (k = 0; k < n; k++)
    {
        w [k] = W[k].w ;
        h [k] = W[k].h ;
        pi[k] = W[k].pi;
    }
    GRB_TRY (GrB_Vector_new(pd_output,  GrB_FP64,   n));
    GRB_TRY (GrB_Vector_new(ppi_output, GrB_UINT64, n));
    GRB_TRY (GrB_Vector_new(ph_output,  GrB_UINT64, n));
    GRB_TRY (GrB_Vector_build (*pd_output , I, w , n, GrB_MIN_FP64  ));
    GRB_TRY (GrB_Vector_build (*ppi_output, I, pi, n, GrB_MIN_UINT64));
    GRB_TRY (GrB_Vector_build (*ph_output , I, h , n, GrB_MIN_UINT64));
    LG_FREE_WORK;
    return (GrB_SUCCESS) ;
}
```

C MASK IMAGES

We interpreted the following images from “Digital Image Processing” [46] as masks:

FigP1012.png

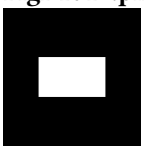
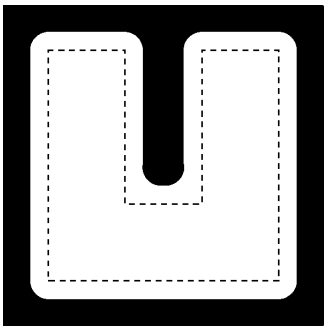
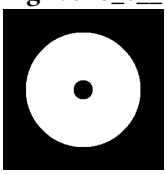


Fig1008_a__step_edge_.png

FigP0905_d_.png



FigP0528_c__doughnut_.png



FigP0616_b_.png

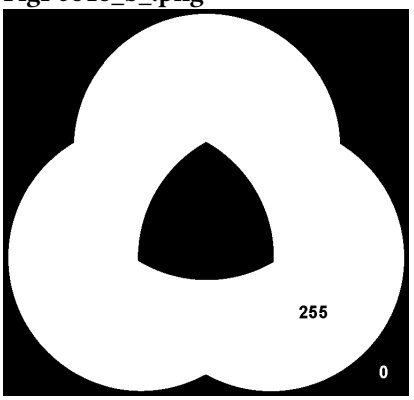
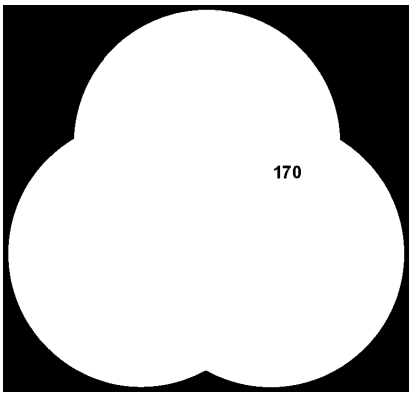
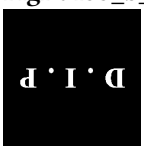


Fig0114_c__bottles_.png

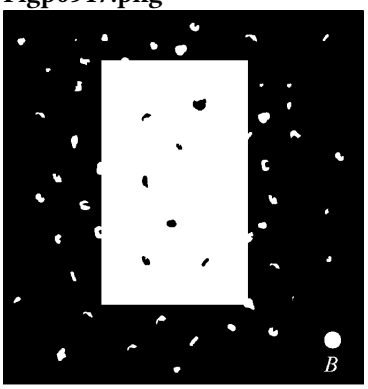
FigP0616_c_.png



FigP0433_b_.png



Figp0917.png



FigP0905_b_.png

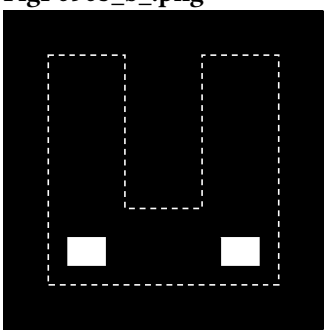


Fig1059_c__NegADI_.png

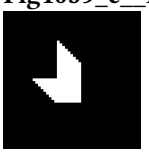


Fig1111_a__triangle_.png

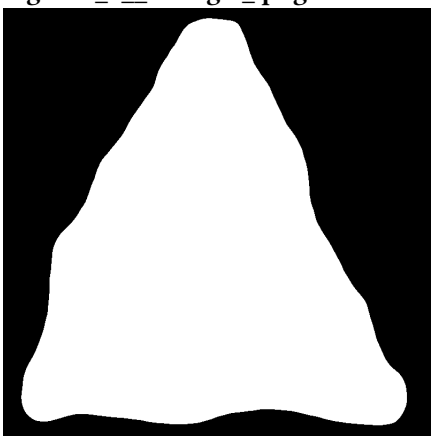
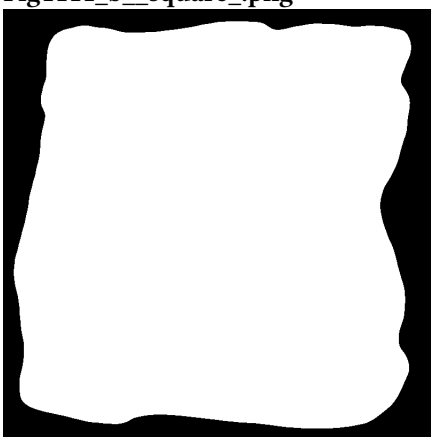
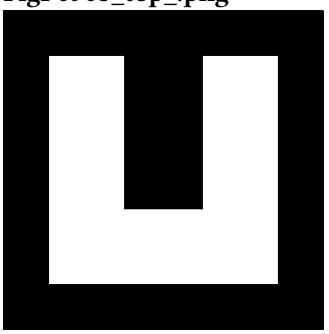


Fig1111_b__square_.png



FigP0905_top_.png



FigP1110.png

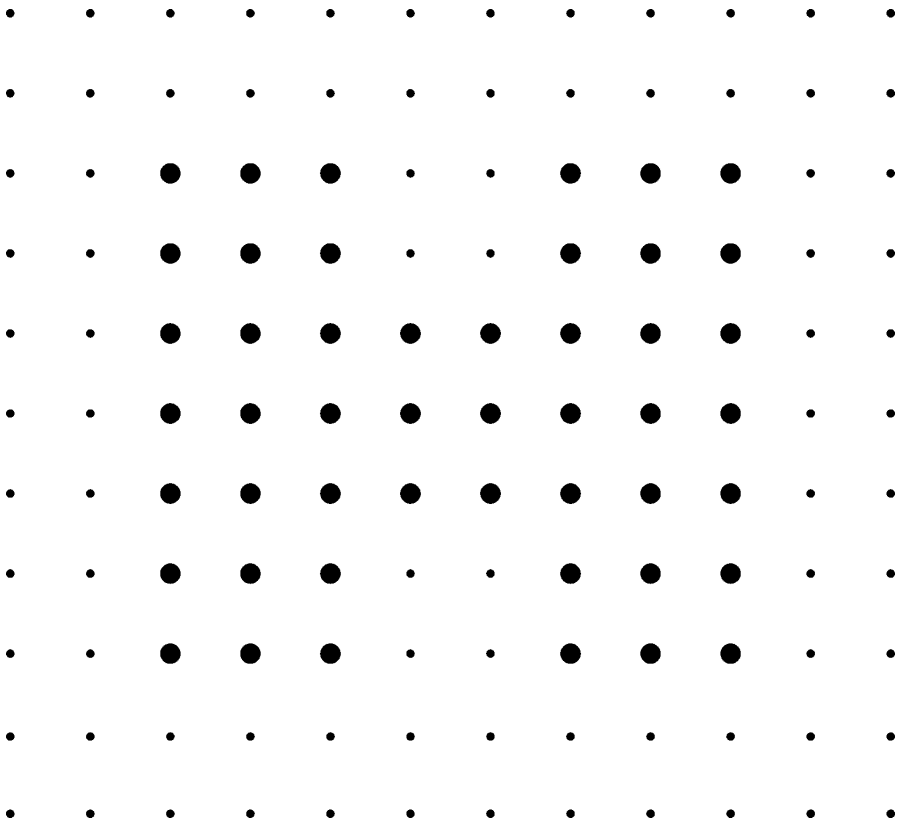
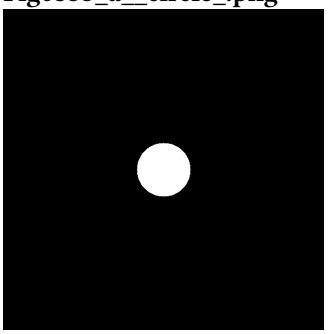
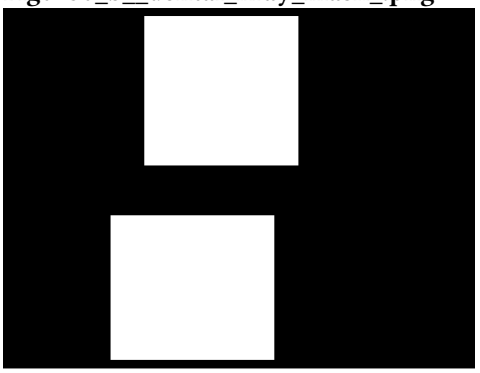


Fig0533_a__circle_.png



FigP0917_noisy_rectangle_.png

Fig0230_b__dental_xray_mask_.png



FigP0528_b__two_dots_.png

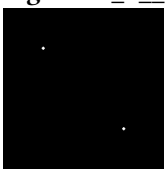


Fig1059_a__AbsADI_.png

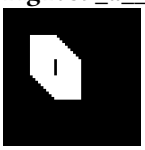
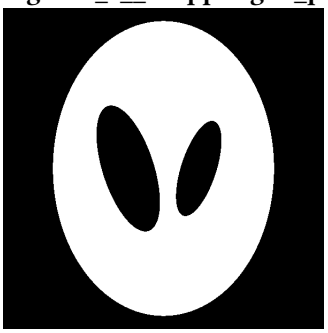


Fig1059_b__PosADI_.png

Fig0539_c__shepp-logan_phantom_.png



FigP0905_c_.png

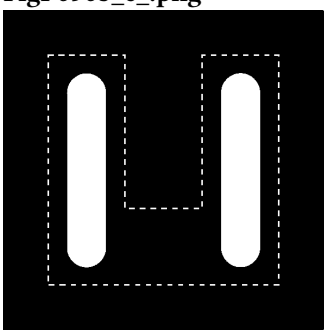
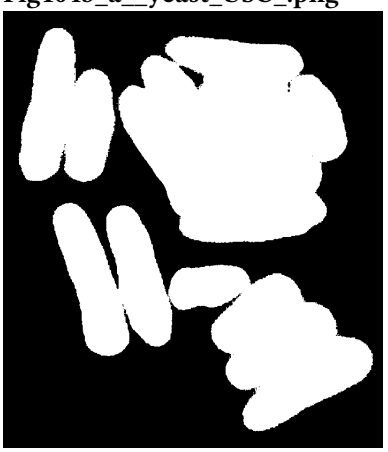


Fig1043_a__yeast_USC_.png



FigP0905_U_.png

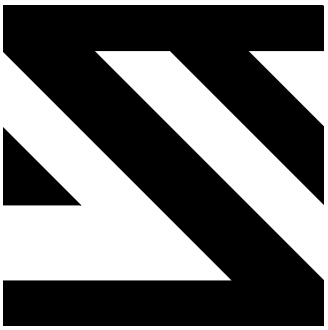


Fig0524_b__blurred-impulse_.png

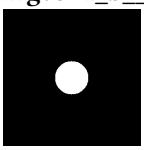


Fig0424_a__rectangle_.png

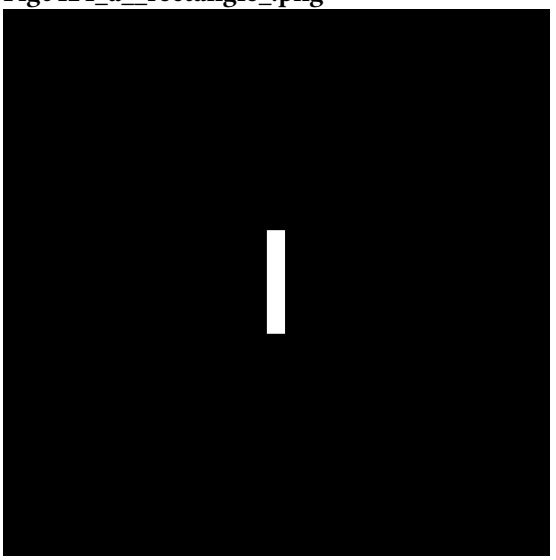
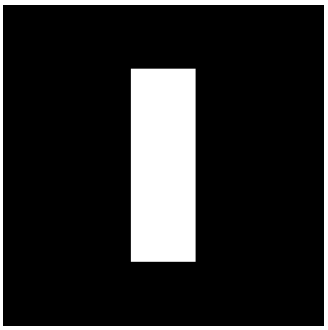
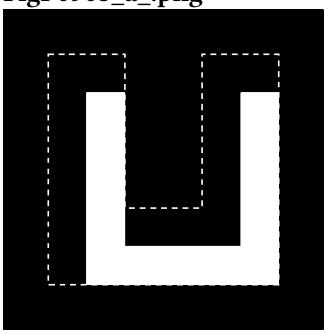


Fig1008_c__roof_edge_.png

Fig0539_a__vertical_rectangle_.png

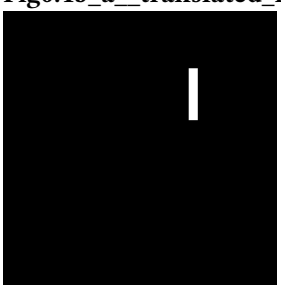


FigP0905_a_.png



FigP0433_a_.png

Fig0.15_a__translated_rectangle_.png



FigP0918_c_.png

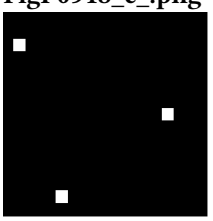


Fig0524_a__impulse_.png

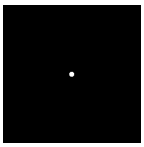


Fig0236_a__letter_T_.png

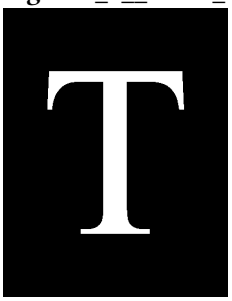
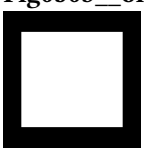


Fig0503__original_pattern_.png



FigP0501.png

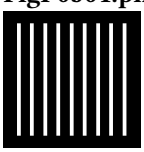


Fig1218_airplanes_.png

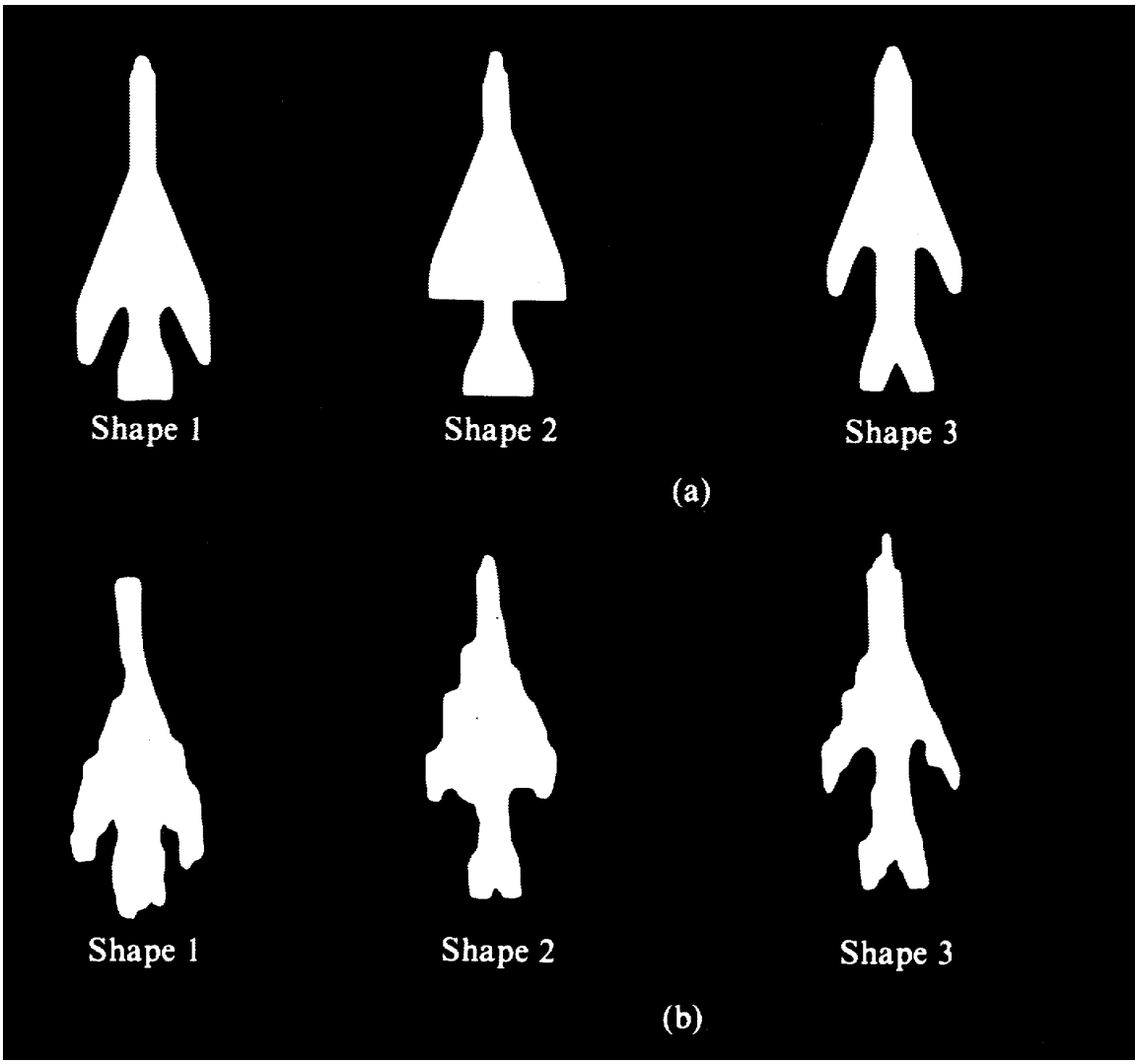
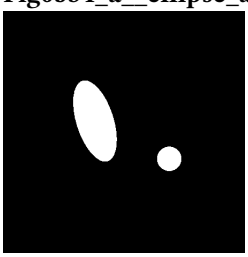
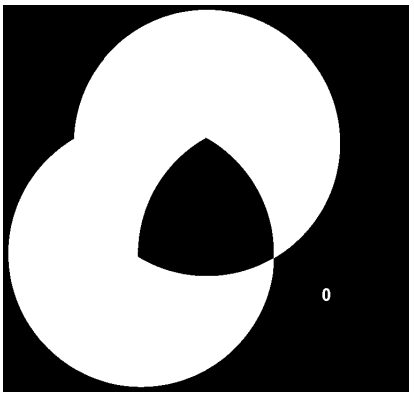


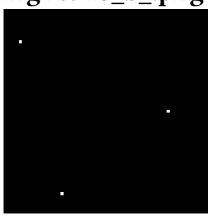
Fig0534_a__ellipse_and_circle_.png



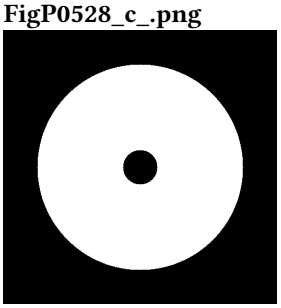
FigP0616_a_.png



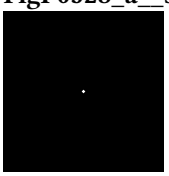
FigP0918_b_.png



FigP0528_c_.png



FigP0528_a__single_dot_.png



Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009